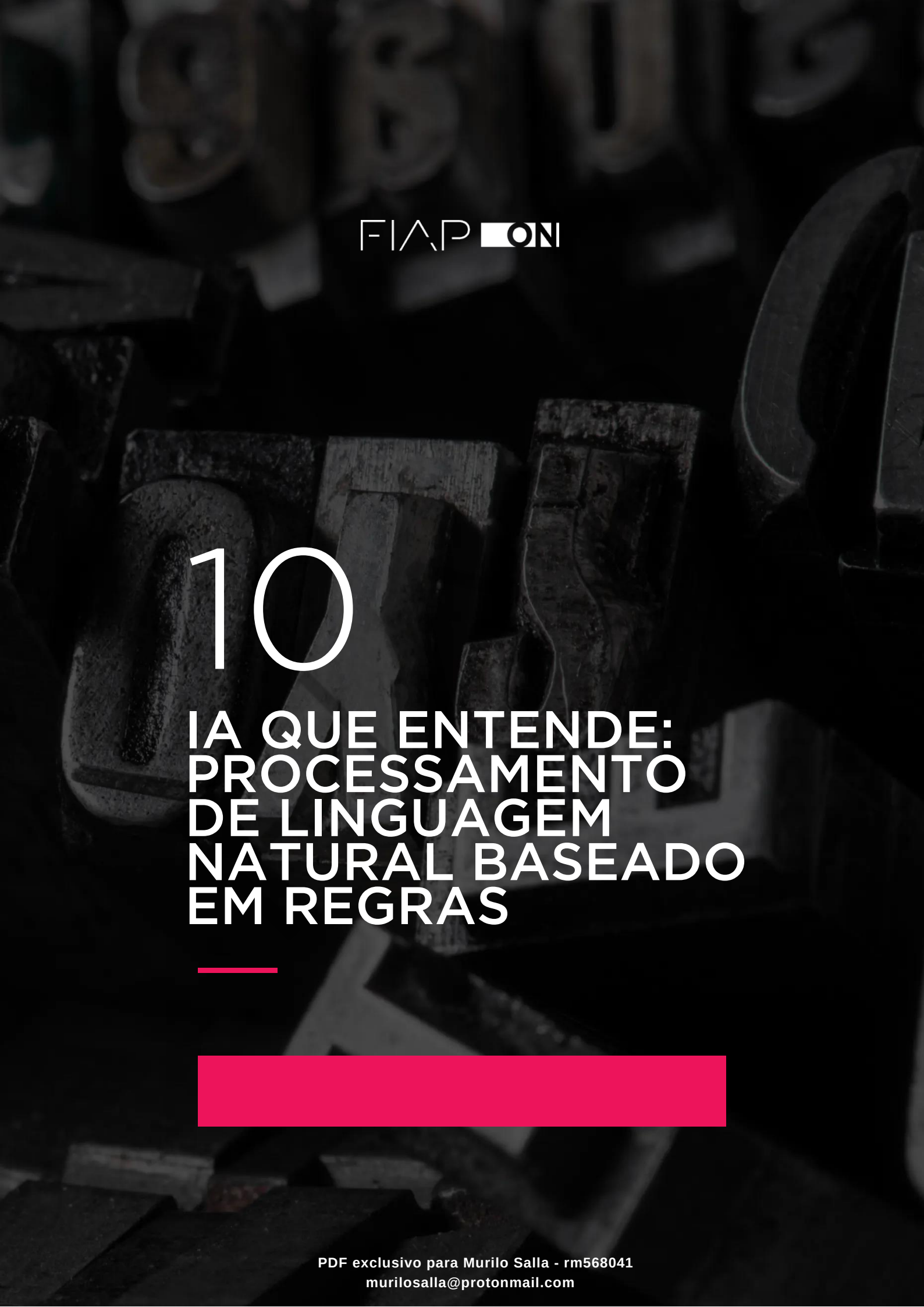


10

IA QUE ENTENDE: PROCESSAMENTO DE LINGUAGEM NATURAL BASEADO EM REGRAS

PDF exclusivo para Murilo Salla - rm568041
murilosalla@protonmail.com

LISTA DE FIGURAS

Figura 1 - Saída do código-fonte 3, atributos das palavras com spaCy 18
Figura 2 - Saída do código-fonte 4, classes gramaticais das palavras com spaCy...27
Figura 3 - Saída do código-fonte 5, árvore sintática com CFG do NTLK 31
Figura 4 - Saída do código-fonte 6, árvore de dependência com spaCy.....33
Figura 5 - Saída do código-fonte 7, imprimindo os descendentes e ancestrais de cada token.....34
Figura 6 - Saída do código-fonte 8, demonstração de ambiguidade sintática com CFG usando NLTK.....36
Figura 7 - Saída do código-fonte 9, demonstração de ambiguidade sintática com spaCy.....36
Figura 8 - Saída do Código-fonte 12, demonstração NER com NLTK 43
Figura 9 - Saída do Código-fonte 13, demonstração NER por Aprendizado de Máquina com spaCy.....46
Figura 10 - Saída do código-fonte 15, ontologia para domínio esportivo 57
Figura 11 - Saída do código-fonte 16, integração entre NER e ontologia 58
Figura 12 - Exemplo de aplicação de web semântica na construção de páginas HTML 60

FIAP

LISTA DE QUADROS

Quadro 1 - Atributos das palavras (tokens) após a análise, léxica, sintática e morfológica.....17
Quadro 2 - Possíveis valores do atributo POS.....26
Quadro 3 - Entidades treinadas no módulo de NER do spaCy 47

FIAP

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Uso do estemizador Snowball do NLTK em Python 15
Código-fonte 2 - Uso do estemizador RSLP do NLTK em Python 15
Código-fonte 3 - Demonstração da análise morfológica no spaCy em Python.....18
Código-fonte 4 - Uso de POS com o spaCy em Python.....27
Código-fonte 5 - Demonstração de um analisador léxico com NLTK em Python.....30
Código-fonte 6 - Análise de dependência com o spaCy em Python.....32
Código-fonte 7 - Complemento do código-fonte 6 para imprimir descendentes e ancestrais.....33
Código-fonte 8 - Demonstração de um analisador léxico com NLTK em Python.....36
Código-fonte 9 - POS tagging e dependências sintáticas com spaCy 36
Código-fonte 10 - Sistema NER com regex em Python 41
Código-fonte 11 - Sistema NER com análise sintática usando spaCy 42
Código-fonte 12 - Sistema NER com análise sintática rasa usando NLTK 43
Código-fonte 13 - Sistema NER com Aprendizado de Máquina usando o spaCy.....46
Código-fonte 14 - Ontologia criada em RDF/OWL para o domínio esportivo do tipo futebol 55
Código-fonte 15 - Abrindo uma ontologia com o rdflib em Python 56
Código-fonte 16 - Validando entidades extraídas com NER do spaCy através de uma ontologia.....58

FIAP

LISTA DE COMANDOS DE PROMPT DO SISTEMA OPERACIONAL

Comando de prompt 1 - Instalação do rdflib 56

FIAP

SUMÁRIO

1 ABORDAGEM SIMBÓLICA EM NLP 7
1.1 O que é a abordagem simbólica? 7
1.2 Quais são as aplicações de sistemas simbólicos em NLP? 7
1.3 Análise léxica, morfológica, sintática e semântica 8

2 ANÁLISE MORFOLÓGICA 11
2.1 Análise morfológica no português 11
2.2 Normalização de palavras 13
2.2.1 Estemização (*stemming*) 14
2.2.2 Lematização (*lemmatization*) 16
2.3 Ambiguidade léxica e morfológica 19

3 ANÁLISE SINTÁTICA 21
3.1 Análise sintática no português 21
3.2 Analisadores sintáticos 24
3.2.1 Etiquetagem morfossintática 24
3.2.2 Análise sintática com gramática livre de contexto 28
3.2.3 Análise de dependência 31
3.3 Ambiguidade sintática 34

4 RECONHECIMENTO DE ENTIDADES NOMEADAS 38
4.1 O que é o reconhecimento de entidades nomeadas? 38
4.2 NER por abordagem simbólica 38
4.2.1 NER por regex 40
4.2.2 NER por análise sintática 41
4.2.3 NER por chucking 42
4.3 NER por meio da abordagem de Aprendizado de Máquina 44

5 ONTOLOGIAS 48
5.1 O que são ontologias? 48
5.2 Construindo ontologias 50
5.3 Integrando ontologias e ferramentas de NLP 57
5.4 Web semântica 59

REFERÊNCIAS 62

FINAL

1 ABORDAGEM SIMBÓLICA EM NLP

1.1 O que é a abordagem simbólica?

A abordagem simbólica em Processamento de Linguagem Natural (NLP) fundamenta-se na representação explícita de conhecimento linguístico por meio de regras formais, gramáticas e dicionários estruturados, refletindo uma visão lógica e estruturada da linguagem humana. Diferente das abordagens estatísticas ou conexionistas, que aprendem padrões a partir de grandes volumes de dados, a abordagem simbólica enfatiza a interpretabilidade, permitindo que sistemas sejam construídos com base em regras sintáticas, semânticas e pragmáticas previamente definidas por especialistas.

Essa abordagem se mostra particularmente valiosa em contextos que exigem alto grau de controle, explicabilidade e consistência, como sistemas jurídicos, médicos ou qualquer domínio onde decisões precisam ser auditáveis. No entanto, sua principal limitação reside na dificuldade de escalar para a complexidade e variabilidade da linguagem natural, exigindo manutenção constante e elevado nível de expertise para incorporar novas expressões, exceções e ambiguidades linguísticas. Ainda assim, em muitos casos, ela serve como uma fundação robusta ou como complemento crucial a métodos híbridos, nos quais regras simbólicas se integram a técnicas estatísticas ou baseadas em Aprendizado de Máquina, unindo o rigor lógico ao poder de generalização dos dados.

1.2 Quais são as aplicações de sistemas simbólicos em NLP?

A abordagem simbólica em NLP é amplamente aplicada em sistemas e tarefas onde a precisão, a interpretabilidade e a governança são fundamentais. Alguns exemplos são:

- **Sistemas baseados em regras gramaticais (gramáticas formais):** corretores gramaticais (*spell checkers*) em processadores de texto como o Microsoft Word e Grammarly. Esses sistemas realizam análise gramatical, correção ortográfica e sintática. Para isso, eles utilizam gramáticas formais

(como gramáticas livres de contexto) e dicionários estruturados para detectar erros e sugerir correções.

- **Analisadores sintáticos e morfológicos (parsers):** são sistemas que realizam a análise sintática de sentenças, identificação de estruturas como sujeito, verbo, objeto, entre outros. Esses sistemas são baseados em regras linguísticas explícitas para decompor frases em suas partes constituintes.
- **Sistemas de inferência lógica:** são sistemas de Inteligência Artificial que utilizam lógica formal e linguagem natural para solucionar problemas, aplicando raciocínio dedutivo a partir de uma base de conhecimento explícita. Para construir a base de conhecimento utilizam-se ontologias, enquanto os sistemas de inferência se baseiam em lógica de predicados e regras semânticas para responder perguntas.
- **Agentes conversacionais baseados em regras:** temos como exemplos o ELIZA (um dos primeiros chatbots, da década de 1960) e os sistemas de atendimento automático baseados em scripts pré-definidos. Nesses sistemas, o diálogo com o usuário ocorre com base em regras fixas, que são utilizadas no mapeamento entre padrões de entrada e respostas predefinidas.
- **Reconhecimento de entidades nomeadas (NER) com base em dicionários:** são módulos utilizados para a identificação de termos técnicos, como nomes de doenças, medicamentos ou procedimentos, palavras especiais e palavras compostas como número de CEP, CPF e RG. Podem ser utilizados em sistemas conversacionais ou em ferramentas de mineração de texto.

1.3 Análise léxica, morfológica, sintática e semântica

A **análise léxica** é focada no reconhecimento e na categorização das unidades mínimas de significado, ou seja, as palavras. Ela envolve a identificação das palavras de uma frase e sua associação a categorias gramaticais, como substantivos, verbos, adjetivos, advérbios, entre outros. Esse processo é realizado com base em um dicionário de palavras e suas respectivas classes gramaticais. A análise léxica pode

ser considerada como a “decodificação” de uma sequência de caracteres em palavras reconhecíveis e suas funções dentro de uma estrutura linguística.

Por outro lado, a **análise morfológica** envolve a decomposição das palavras em suas unidades mínimas de significado, chamadas morfemas. Morfemas são os menores elementos da língua que carregam significado e podem ser, por exemplo, um radical, uma raiz ou uma afixação (prefixo e sufixo). A análise morfológica tem como objetivo identificar e classificar esses morfemas em palavras, ajudando a compreender melhor a estrutura interna das palavras e seu comportamento dentro de uma frase.

Em NLP, a análise morfológica é o processo de segmentação de palavras para identificar suas componentes morfológicas e classificar as palavras conforme suas categorias gramaticais. Esse processo é fundamental para entender como as palavras são formadas e para ajudar em tarefas como a desambiguação (quando a mesma palavra tem significados diferentes) e a normalização de palavras.

Já a **análise sintática** lida com a estrutura da sentença em um nível superior, uma vez que as palavras já foram identificadas e classificadas. Ela tem como objetivo compreender como essas palavras se combinam para formar frases e orações gramaticais.

A análise sintática constrói uma árvore sintática que representa a estrutura hierárquica da frase, destacando como os elementos da frase — como o sujeito, o predicado e os complementos — se relacionam entre si, com base em regras gramaticais. Em outras palavras, enquanto a análise léxica é responsável por “identificar” as palavras e suas classes gramaticais, a análise sintática organiza essas palavras de acordo com sua função e posição na estrutura da sentença.

Por fim, a **análise semântica** é uma fase posterior que busca atribuir significado às sentenças ou expressões identificadas. Enquanto a análise sintática se preocupa com a estrutura da linguagem, a análise semântica visa interpretar o conteúdo dessa estrutura. Ela tem o desafio de associar as relações entre palavras e frases a conceitos ou entidades do mundo real, algo que não é trivial, dada a natureza ambígua e flexível da linguagem humana.

Vale ressaltar que as diferentes ambiguidades linguísticas podem comprometer tarefas em NLP. A ambiguidade **léxica** ocorre quando uma mesma palavra tem

múltiplos significados, como “banco”, que pode indicar uma instituição financeira ou um assento. A **morfológica** é oriunda de uma ambiguidade léxica quando não é possível identificar a classe gramatical de uma palavra, como “canto”, que pode ser verbo (do verbo cantar) ou substantivo (esquina ou local). Já a ambiguidade **sintática** surge quando uma sentença permite mais de uma estrutura gramatical possível, como em “vi o homem com o telescópio”, que pode significar que o observador estava utilizando o telescópio ou que o homem observado o carregava.

Por fim, todas essas ambiguidades podem levar à ambiguidade **semântica**, que está relacionada à interpretação do significado da frase como um todo — mesmo quando a estrutura sintática parece clara.

FIAP

2 ANÁLISE MORFOLÓGICA

2.1 Análise morfológica no português

A análise morfológica é uma parte da gramática que se dedica ao estudo das palavras isoladamente, classificando-as de acordo com a sua classe gramatical — também chamada de classe morfológica ou categoria lexical.

Essa análise ignora, em um primeiro momento, as relações sintáticas que as palavras estabelecem entre si em uma frase, concentrando-se apenas na forma, estrutura e função de cada palavra como unidade. A análise morfológica se preocupa em classificar e descrever a forma da palavra — sua classe gramatical, flexões, partes que a compõem, como radical, afixos, desinências.

Análises léxica e morfológica, embora estejam relacionadas, pertencem a níveis diferentes do estudo linguístico, com focos distintos e complementares. A análise léxica está ligada ao estudo do **léxico** de uma língua, ou seja, ao conjunto de palavras e expressões que compõem **o vocabulário de um idioma**. Ela investiga o significado, a origem, a formação e o uso das palavras, sendo uma área que se aproxima da **lexicologia** (estudo das palavras) e da **semântica** (estudo dos significados).

No português, existem dez classes gramaticais, sendo que algumas são variáveis — ou seja, sofrem flexões de gênero, número, tempo, modo —, e outras invariáveis. As classes variáveis incluem o substantivo, o adjetivo, o artigo, o numeral, o pronome e o verbo. Já as invariáveis são o advérbio, a preposição, a conjunção e a interjeição.

Cada classe apresenta características morfológicas próprias. O substantivo é a palavra que nomeia seres, coisas, lugares, sentimentos, entre outros, e pode variar em gênero e número. O adjetivo é a palavra que qualifica ou caracteriza o substantivo, concordando com ele em gênero e número. O verbo indica ações, estados ou fenômenos e possui uma rica flexão que envolve tempo, modo, pessoa e número. Os pronomes substituem ou acompanham os substantivos, variando conforme a função que desempenham. O artigo determina o substantivo, enquanto os numerais indicam quantidade ou ordem.

Já as palavras invariáveis não se flexionam. O advérbio modifica o verbo, o adjetivo ou outro advérbio, indicando circunstâncias diversas, como tempo, modo e lugar. A preposição, por sua vez, estabelece relações entre palavras, geralmente ligando termos da oração. A conjunção interliga orações ou termos semelhantes, e, finalmente, a interjeição expressa emoções ou reações de maneira espontânea.

O estudo da estrutura raiz das palavras — ou seja, o reconhecimento do **radical** — insere-se no campo da morfologia, mais especificamente, na chamada **morfologia lexical**. Essa área da gramática analisa a estrutura interna das palavras, identificando seus elementos constituintes: radical, afixos (prefixos e sufixos), vogal temática, desinências e outras partículas formadoras. O radical é a base significativa da palavra, a parte que contém o sentido essencial e comum a uma família de palavras derivadas.

Por exemplo, na palavra “felicidade”, o radical é "*felic*-", que também aparece em “feliz”, “infeliz”, “felicitar” e “felicíssimo”. Mesmo com diferentes afixos e desinências, o radical conserva a ideia central relacionada à noção de felicidade. Identificar esse elemento permite compreender não só a origem e a formação das palavras, mas também estabelecer relações entre elas, o que enriquece o vocabulário e a compreensão linguística.

O estudo dos radicais, portanto, é essencialmente morfológico pois trata da formação e estrutura das palavras. Ele investiga como novas palavras surgem a partir de elementos comuns e como os significados se mantêm ou se transformam por meio dos processos de derivação e composição. Mesmo que o radical tenha implicações semânticas — ou seja, de significado —, seu reconhecimento e análise fazem parte da morfologia, pois dizem respeito à forma e à constituição morfológica do vocábulo.

Você pode estar se perguntando se a forma raiz das palavras tem uma origem histórica no desenvolvimento natural de uma língua. Essa pergunta é estudada por uma disciplina própria denominada **filologia**, que combina elementos de linguística, literatura e história.

Enquanto a morfologia analisa a estrutura atual das palavras e a análise léxica foca no significado e uso atual, a filologia investiga a origem e o percurso histórico das palavras e da própria língua. Logo, para entender a fundo cada palavra, não basta apenas conhecer sua estrutura e o seu significado, mas também sua origem, evolução e uso ao longo do tempo, dentro de um contexto histórico, literário e cultural.

Por sorte, para nossos propósitos no estudo de NLP, não precisaremos entrar nesse nível de detalhe sobre as palavras. Nosso interesse reside em estabelecer regras que permitam atribuir diferentes relações de forma para uma mesma palavra, em uma etapa conhecida como lematização e estemização. Vamos nos aprofundar nesses conceitos a seguir.

2.2 Normalização de palavras

A normalização de palavras é uma etapa fundamental no processamento de linguagem natural, cujo objetivo é reduzir a variabilidade linguística e tornar os dados textuais mais uniformes e manejáveis por algoritmos computacionais. Trata-se de um processo que atua sobre as diferentes formas que uma mesma palavra pode assumir, com o intuito de representá-las de maneira padronizada, preservando seu significado essencial. Dentro desse escopo, a análise morfológica desempenha um papel fundamental, pois é ela que fornece as bases linguísticas para que essa transformação ocorra de maneira coerente.

A análise morfológica, no contexto da NLP, consiste em decompor as palavras em seus elementos constituintes — como radical, afixos e desinências — e identificar suas categorias gramaticais (substantivo, verbo, adjetivo, entre outros) e propriedades morfológicas (tempo, número, gênero, pessoa). Esse entendimento da estrutura interna das palavras permite reconhecer que formas distintas, como “amamos”, “amará” e “amando”, todas se referem ao mesmo verbo “amar”, embora estejam flexionadas de maneiras diferentes.

É a partir desse conhecimento que se aplicam as técnicas de **normalização**, como a **estemização** (*stemming*) e a **lematização** (*lemmatization*). A estemização busca extrair uma raiz comum entre as palavras, muitas vezes ignorando regras gramaticais mais complexas; já a lematização vai além, utilizando o resultado da análise morfológica para identificar o **lema** (ou forma base), que corresponde à entrada de dicionário da palavra. Assim, em vez de tratar “corri”, “corremos” e “correndo” como palavras diferentes, o sistema passa a reconhecê-las todas como instâncias do verbo “correr”.

A normalização depende diretamente de uma análise morfológica eficaz. Sem compreender as regras de formação e flexão das palavras, os algoritmos não

conseguiriam agrupar corretamente formas derivadas, nem distinguir entre homônimos ou variantes que só fazem sentido dentro de um determinado contexto gramatical.

Em línguas ricas em morfologia, como o português, essa etapa torna-se ainda mais crítica, uma vez que a complexidade flexional pode gerar milhares de formas diferentes para um mesmo termo base.

2.2.1 Estemização (*stemming*)

Entre os algoritmos de estemização mais conhecidos e utilizados está o **Snowball**, desenvolvido por Martin Porter como uma evolução do seu próprio algoritmo original, o Porter Stemmer. O Snowball se destaca por sua abordagem linguística adaptada a diversos idiomas, incluindo o português, oferecendo regras específicas para lidar com os sufixos e com as particularidades de cada língua.

Por meio da aplicação de um conjunto de regras iterativas baseadas na remoção de sufixos, o Snowball é capaz de extrair de forma eficiente a raiz das palavras, ainda que o resultado nem sempre corresponda a uma palavra válida do vocabulário, já que o foco está na simplificação para agrupar termos semelhantes.

Além do Snowball, existem outros algoritmos populares de estemização, como o próprio Porter Stemmer, que foi pioneiro no campo e ainda é bastante utilizado em inglês, e o algoritmo de estemização de Lovins, conhecido por ser um dos primeiros e mais agressivos em termos de remoção de sufixos. Também há abordagens mais simples, como o estemizador RSLP (Removedor de Sufixos da Língua Portuguesa), especificamente desenvolvido para o português, que utiliza regras de remoção de sufixos comuns na língua.

No código-fonte a seguir, apresentamos um exemplo de uso do Snowball em português, por meio da biblioteca NLTK:

```python
from nltk.stem.snowball import SnowballStemmer

stemmer = SnowballStemmer("portuguese")

palavras = ["felicidade", "felizmente", "feliz", "felizardo",
"correndo", "correr"]

for palavra in palavras:
    print(f"{palavra} -> {stemmer.stem(palavra)}")
```

Código-fonte 1 - Uso do estemizador Snowball do NLTK em Python
Fonte: Elaborado pelo autor (2025)

Também podemos utilizar outro estemizador para realizar a normalização. No próprio NLTK há o módulo RSLP. Vejamos o exemplo de uso:

```python
#import nltk # Caso não rode, você deve baixar o rslp
#nltk.download('rslp')

from nltk.stem import RSLPStemmer

stemmer = RSLPStemmer()

palavras = ["felicidade", "felizmente", "feliz", "felizardo",
"correndo", "correr"]

for palavra in palavras:
    print(f"{palavra} -> {stemmer.stem(palavra)}")
```

Código-fonte 2 - Uso do estemizador RSLP do NLTK em Python
Fonte: Elaborado pelo autor (2025)

Observe que, eventualmente, será necessário baixar o módulo RSLP apagando as anotações das primeiras duas linhas. Além disso, para as palavras usadas nos dois exemplos, o resultado é o mesmo. Repita ambos os códigos trocando para: `palavras = ["amar", "amador", "amando", "amou", "ama", "amo"]`. Os resultados obtidos continuam iguais para ambos os estemizadores?

Embora a estemização seja um processo rápido e eficiente, pode gerar ambiguidades e raízes truncadas que não correspondem a formas léxicas reais, o que pode limitar sua precisão em determinadas aplicações. Por isso, em casos que demandam de maior rigor linguístico, a estemização pode ser complementada ou substituída pela lematização, que busca a forma canônica da palavra com base em dicionários e análise gramatical.

Ainda assim, a estemização é uma técnica valiosa e amplamente utilizada por sua simplicidade e velocidade, especialmente em sistemas em que a normalização textual eficiente é uma priorioridade.

2.2.2 Lematização (*lemmatization*)

Diferente da estemização, que muitas vezes realiza uma simples remoção de sufixos para extrair a raiz da palavra, a lematização envolve uma análise mais profunda e linguística, considerando o contexto gramatical, a categoria da palavra e suas propriedades morfológicas. Por exemplo, as palavras “corri”, “correndo” e “corremos” possuem formas superficiais diferentes, mas sua lematização correta aponta para o lema “correr”.

Esse processo depende de dicionários, regras morfológicas e, em alguns casos, do contexto sintático para garantir que a forma base seja adequada ao uso. A lematização é particularmente importante para idiomas com alta flexão, como o português, pois permite agrupar variações verbais, nominais e adjetivais, simplificando a análise semântica e a recuperação de informação.

Além disso, a lematização contribui para reduzir a dispersão dos dados, tornando os modelos de Aprendizado de Máquina mais eficientes ao tratar palavras relacionadas como instâncias de um mesmo conceito. Em contrapartida, esse processo exige maior custo computacional e um nível mais sofisticado de análise em comparação à estemização, que é mais simples e rápida, porém menos precisa.

A principal diferença entre o lematizador do NLTK e o do spaCy está na abordagem e no nível de precisão com que realizam a tarefa de reduzir palavras à sua forma base.

O lematizador do NLTK, principalmente o WordNetLemmatizer, baseia-se no dicionário WordNet e depende de uma lista de palavras e suas formas canônicas, funcionando melhor para o inglês. Ele exige que a categoria gramatical (POS tag) seja informada manualmente para garantir uma lematização mais correta, o que pode limitar sua precisão caso essa informação não seja adequada. Além disso, o NLTK não possui suporte nativo robusto para o português, o que dificulta sua aplicação em línguas com morfologia mais complexa.

Já o spaCy oferece um lematizador integrado a um pipeline linguístico que combina modelos estatísticos e regras morfológicas específicas para cada idioma, incluindo o português. Ele também realiza a análise morfológica completa e considera o contexto sintático para determinar o lema correto automaticamente, sem a necessidade de entrada manual das categorias gramaticais.

No quadro a seguir, apresentamos alguns dos principais atributos de uma palavra (token) que é retornada pela biblioteca spaCy.

Nota: perceba que o spaCy ao analisar uma palavra, realiza a análise léxica, morfológica e sintática de uma só vez, estabelecendo todos os atributos de cada token processado no texto.

Atributo	Descrição	Exemplo no spaCy
LEMMA	Forma raiz da palavra.	token.lemma_
POS	Parte da fala da palavra (classe gramatical).	token.pos_
TAG	Expressa a POS com informações morfológicas da palavra.	token.tag_
DEP	Dependência sintática entre as palavras.	token.dep_
SHAPE	Formato das palavras (maiúscula, minúscula, pontuação, dígitos).	token.shape_
MORPH	Atributos morfológicos finos como tempo verbal, pessoa, gênero.	token.morph
ALPHA	Se a palavra é de caracteres alfabéticos (lembre-se que há tokens que não são palavras tradicionais como o espaço e o início de frase).	token.is_alpha
STOP	Se a palavra é do tipo stopword.	token.is_stop

Quadro 1 - Atributos das palavras (tokens) após a análise, léxica, sintática e morfológica
Fonte: Raj (2019), adaptado pelo autor (2025)

No código-fonte a seguir, demonstramos o uso do spaCy para extrair os atributos morfológicos das palavras, possibilitando sua lematização. Já o resultado da execução do código é apresentado na figura “Saída do código-fonte 3, atributos das palavras com spaCy”.

```python
#!python -m spacy download pt_core_news_sm
import spacy

nlp = spacy.load("pt_core_news_sm")

texto = "A inteligência artificial, aliada ao Processamento de Linguagem Natural (NLP), transforma dados em compreensão, permitindo que máquinas entendam e se comuniquem com os humanos de forma cada vez mais natural."

doc = nlp(texto)

# Exibe os atributos principais para cada token relevante
print(f"{'TOKEN':<20}{'LEMMA':<20}{'MORPH'}")
print("=" * 60)
for token in doc:
  if not token.is_punct and not token.is_space:
    print(f"{token.text:<20}{token.lemma_:<20}{token.morph}")
```

Código-fonte 3 - Demonstração da análise morfológica no spaCy em Python
Fonte: Elaborado pelo autor (2025)

```
TOKEN               LEMMA               MORPH
============================================================
A                   o                   Definite=Def|Gender=Fem|Number=Sing|PronType=Art
inteligência        inteligência        Gender=Fem|Number=Sing
artificial          artificial          Gender=Fem|Number=Sing
aliada              aliar               Gender=Fem|Number=Sing|VerbForm=Part
ao                  a o                 Definite=Def|Gender=Masc|Number=Sing|PronType=Art
Processamento       Processamento       Gender=Masc|Number=Sing
de                  de
Linguagem           Linguagem           Number=Sing
Natural             Natural             Number=Sing
NLP                 NLP                 Gender=Fem|Number=Sing
transforma          transformar         Mood=Ind|Number=Sing|Person=3|Tense=Pres|VerbForm=Fin
dados               dar                 Gender=Masc|Number=Plur
em                  em
compreensão         compreensão         Gender=Fem|Number=Sing
permitindo          permitir            VerbForm=Ger
que                 que
máquinas            máquina             Gender=Fem|Number=Plur
entendam            entendar            Mood=Ind|Number=Plur|Person=3|Tense=Pres|VerbForm=Fin
e                   e
se                  se                  Case=Acc|PronType=Prs
comuniquem          comunicar           Mood=Sub|Number=Plur|Person=3|Tense=Pres|VerbForm=Fin
com                 com
os                  o                   Definite=Def|Gender=Masc|Number=Plur|PronType=Art
humanos             humano              Gender=Masc|Number=Plur
de                  de
forma               forma               Gender=Fem|Number=Sing
cada                cada                Gender=Masc|Number=Sing
vez                 vez
mais                mais
natural             natural             Gender=Masc|Number=Sing
```

Figura 1 - Saída do código-fonte 3, atributos das palavras com spaCy
Fonte: Elaborado pelo autor (2025)

Importante: a divisão análise léxica, morfológica e sintática segue um objetivo didático. Alguns códigos de NLP podem realizar todas essas tarefas ou partes intercaladas em um só momento, não havendo uma distinção tão clara na execução.

2.3 Ambiguidade léxica e morfológica

A ambiguidade léxica em português pode surgir tanto de palavras **homógrafas** quanto de palavras **polissemânticas**, dificultando o processamento automático da linguagem.

Palavras homógrafas são aquelas que têm a mesma grafia, mas significados distintos e, às vezes, pronúncia diferente. Um exemplo clássico é a palavra “manga”, que pode significar tanto a fruta tropical quanto a parte da camisa onde se encaixa o braço, dependendo do contexto. Da mesma forma, “banco” pode se referir a uma instituição financeira ou a um assento, como em “fui ao banco resolver um problema” *versus* “sentei no banco da praça”.

Já as palavras polissêmicas têm um único radical, mas carregam múltiplos significados relacionados por extensão de sentido. Um bom exemplo é “cabeça”, que pode indicar a parte do corpo humano, como em “doeu minha cabeça”, ou representar liderança, como em “ele é a cabeça do grupo”. Outra palavra polissêmica é “ponto”, que pode significar uma marcação em uma superfície, um local de parada de ônibus, uma pontuação em prova ou até um estágio de cozimento, como em “o bife está no ponto”.

Esses exemplos mostram como a ambiguidade léxica, tanto por homografia quanto por polissemia, desafia as técnicas de análise morfológica em NLP, exigindo mecanismos mais avançados de desambiguação baseados em contexto.

A ambiguidade léxica representa um desafio significativo para a análise morfológica (ambiguidade morfológica) em NLP, especialmente em processos como lematização e estemização. A estemização, por reduzir palavras a suas raízes sem considerar o contexto, pode diminuir a quantidade de termos distintos com formas semelhantes, comprometendo a precisão da análise semântica.

Já a lematização, embora mais sofisticada por buscar o lema com base na classe gramatical e no contexto, ainda enfrenta dificuldades quando o contexto não é

suficientemente claro para desambiguar termos polissêmicos. Assim, a ambiguidade léxica limita a eficácia dessas técnicas, podendo impactar negativamente tarefas subsequentes como a recuperação de informação, análise de sentimentos ou tradução automática.

FIAP

3 ANÁLISE SINTÁTICA

3.1 Análise sintática no português

A análise sintática no contexto da língua portuguesa é o processo linguístico que visa estudar a estrutura das frases e as funções desempenhadas pelas palavras dentro delas. Ela busca identificar como os elementos de uma oração se organizam e como se inter-relacionam para formar o sentido completo e coeso. A análise sintática examina a função de cada palavra ou grupo de palavras em uma frase e a forma como esses componentes se conectam para transmitir uma ideia.

A sintaxe, enquanto parte da gramática, é responsável por determinar as regras que governam a disposição e a concordância dos termos na oração. É essencial compreender que uma oração não é apenas uma combinação aleatória de palavras, mas sim uma organização estruturada e hierarquizada, na qual cada palavra tem uma função específica, sendo essas funções muitas vezes determinadas pela posição das palavras e pelas relações entre elas.

A análise sintática se ocupa principalmente em identificar **sujeito**, **predicado** e seus **componentes**. O sujeito é o termo da oração sobre o qual se faz uma declaração. Ele normalmente é o “ator” da ação, embora não seja sempre o responsável pela ação, como no caso das orações com sujeito indeterminado ou com verbos impessoais. Por sua vez, o predicado é aquilo que se declara sobre o sujeito. Pode ser verbal, nominal ou verbo-nominal, dependendo do tipo de verbo e da estrutura da oração.

Por outro lado, na sintaxe da língua portuguesa, sintagma é um conjunto de palavras que se agrupam dentro de uma oração, com uma estrutura hierarquizada e funcional, e que desempenham uma função específica. Cada sintagma tem um núcleo, que é a palavra principal que determina o tipo de sintagma — e, consequentemente, a sua função na oração —, e pode ser acompanhado de palavras acessórios que expandem ou complementam o sentido. Os sintagmas são fundamentais para a análise sintática, pois ajudam a entender como os elementos de uma oração se organizam e se inter-relacionam.

A seguir, vamos detalhar os principais tipos de sintagmas, suas funções e características:

- **Sintagma Nominal (SN):** o sintagma nominal é centrado em um substantivo ou um pronome, que é o núcleo do sintagma. Ele pode ser simples (apenas o substantivo ou pronome) ou expandido (com adjetivos, artigos, pronomes, entre outros).
  - Exemplo: "A casa grande", em que "casa" é o núcleo do sintagma nominal, e "a" (artigo definido) e "grande" (adjetivo) são os termos que a acompanham.
  - Outros exemplos: "O cachorro", "Aquela escola pública", "Eu" (um pronome pessoal, que também é um sintagma nominal simples).
- **Sintagma Verbal (SV):** o sintagma verbal é centrado no verbo, que é o núcleo do sintagma. Esse tipo de sintagma pode ser composto pelo verbo e seus complementos (objeto direto, objeto indireto, adjuntos adverbiais, entre outros).
  - Exemplo: "Foi ao mercado", em que "foi" é o núcleo do sintagma verbal, e "ao mercado" é o complemento (um sintagma preposicionado).
  - Outros exemplos: "Comi o bolo", "Estudarei amanhã", "Está chorando".
- **Sintagma Adjetivo (SAdj):** o sintagma adjetivo é centrado em um adjetivo. Ele pode atuar como adjunto adnominal, qualificando um substantivo, ou como predicado, atribuindo uma característica ao sujeito ou objeto.
  - Exemplo: "Muito feliz", em que "feliz" é o núcleo do sintagma adjetivo, e "muito" é um modificador do adjetivo.
  - Outros exemplos: "Extremamente cansado" e "Feliz por ter vencido".
- **Sintagma Adverbial (SAdv):** o sintagma adverbial é centrado em um advérbio e tem a função de modificar um verbo, um adjetivo ou outro advérbio, fornecendo informações sobre circunstâncias de tempo, modo, lugar, intensidade, causa, entre outros.
  - Exemplo: "Muito rapidamente", em que "rapidamente" é o núcleo do sintagma adverbial, e "muito" é um modificador de intensidade.

- Outros exemplos: "Aqui perto" e "De manhã".

- **Sintagma Preposicional (SPrep):** o sintagma preposicional é formado por uma preposição seguida de um termo que pode ser um sintagma nominal, um pronome ou até mesmo um verbo no infinitivo. Ele funciona como complemento de outros termos na oração, como objeto de um verbo, de um nome, ou adjunto adverbial.
  - Exemplo: "Na escola", em que "na" (preposição + artigo definido) e "escola" (substantivo) formam o sintagma preposicional.
  - Outros exemplos: "Com ela" e "Para o trabalho".

Cada tipo de sintagma tem uma função específica dentro da oração. Compreender os sintagmas é fundamental para a análise sintática, pois eles permitem visualizar como as palavras se agrupam para formar unidades de sentido.

Ao segmentar uma oração em sintagmas, podemos identificar a função de cada termo de maneira mais clara, além de entender como as palavras se organizam para formar significados mais complexos.

São alguns exemplos de funções que os sintagmas podem desempenhar:

- **Sujeito:** o sintagma nominal pode funcionar como sujeito da oração, ou seja, aquele que realiza ou sofre a ação verbal.
  - Exemplo: "A menina canta".
- **Predicado:** o sintagma verbal pode ser o predicado da oração, isto é, a parte da oração que contém o verbo e, geralmente, os complementos.
  - Exemplo: "Ela viajou para a praia".
- **Objeto:** o sintagma nominal pode ser um objeto direto ou indireto de um verbo.
  - Exemplo: "Eu comprei um presente" (o sintagma "um presente" é o objeto direto de "comprei").
- **Complemento:** sintagmas adjetivos, adverbiais ou preposicionais podem atuar como complementos de outros elementos na oração, como complemento nominal, complemento de verbo ou adjunto adverbial.

- Exemplo: "Ele falou com muita confiança" (o sintagma preposicional "com muita confiança" funciona como adjunto adverbial de modo).

Esses conceitos linguísticos formam a base teórica para o desenvolvimento de algoritmos computacionais de processamento de linguagem natural. Esses algoritmos se inserem na abordagem simbólica à medida que se baseiam nesses formalismos.

Torna-se evidente que a contribuição de linguistas foi essencial nesse desenvolvimento. A seguir, iremos discutir como aplicar esses conceitos e alguns outros relacionados as regras gramaticais que, normalmente, são vistas nas aulas de português e inglês, mas agora serão usados para construir algoritmos poderosos de NLP.

3.2 Analisadores sintáticos

3.2.1 Etiquetagem morfossintática

No contexto da análise sintática, o **Part-of-Speech Tagging** (ou **etiquetagem morfossintática**) é o processo de atribuir a classe gramatical (ou categoria sintática) correta a cada palavra em uma sentença. Isso significa identificar se cada palavra é, por exemplo, um substantivo **(NOUN)**, verbo **(VERB)**, adjetivo **(ADJ)**, advérbio **(ADV)**, preposição **(ADP)**, entre outras categorias linguísticas.

Esse processo é essencial, pois a análise sintática depende do **papel estrutural das palavras na frase**, o que está diretamente ligado à sua classe gramatical. Por exemplo, para que um sistema de NLP compreenda a estrutura de uma oração como “O menino correu rapidamente”, ele precisa saber que: “menino” é um substantivo (sujeito); “correu” é um verbo (predicado); e “rapidamente” é um advérbio (modificador do verbo).

O POS tagging serve como base para etapas posteriores, como *parsing* (análise de dependências ou constituintes), o reconhecimento de entidades nomeadas (NER) e a desambiguação lexical (palavras homógrafas).

Existem dois métodos principais:

- **Metódo 1 – Baseado em regras:** utiliza gramáticas e dicionários para atribuir classes gramaticais, dependendo do contexto da palavra.

- **Método 2 – Baseado em Estatística ou Aprendizado de Máquina:** modelos treinados em corpora anotados (como Campo Aleatório Condicional CRF, Modelo de Markov Oculto HMM, ou redes neurais) que preveem a categoria gramatical com base na sequência de palavras e seus contextos.

Palavras com múltiplas possibilidades de classificação gramatical dependendo do contexto — como “gosto”, que pode ser um substantivo ou uma forma verbal — representam um dos maiores obstáculos para a precisão do etiquetamento. Sistemas baseados em regras frequentemente têm dificuldade em lidar com essas ambiguidades de forma eficiente, enquanto abordagens estatísticas ou baseadas em Aprendizado de Máquina dependem da qualidade e diversidade do corpus de treinamento para reconhecer padrões contextuais sutis.

No módulo spaCy há um analisador sintático aperfeiçoado responsável por realizar o *parser* da frase, atribuindo TAGs a cada uma das palavras. Essas TAGs correspondem a classe gramatical dos tokens, sendo um atributo de cada um deles, assim como a morfologia e o lema discutidos na análise morfológica.

O spaCy adota uma abordagem de Aprendizado Supervisionado, utilizando modelos discriminativos baseados em redes neurais — como Deep Learning — para tarefas como a marcação de classes gramaticais (POS tagging). Diferente de modelos clássicos como HMMs ou CRFs, ele emprega técnicas mais modernas e eficientes, treinadas em grandes corpora anotados.

O quadro a seguir apresenta as principais classes gramaticais que o spaCy identifica.

POS	Descrição	Exemplos
ADJ	Adjetivo	Grande, velho, verde, bonito.
ADP	Adposição (preposição, posposição, circumposições)	Em, para, durante.
ADV	Advérbio	Muito, amanhã, abaixo, onde, lá.
AUX	Verbo auxiliar	É, fez, fará.
CONJ	Conjunção	E, ou, mas.
CCONJ	Conjunção coordenativa	Nem, contudo, todavia, pois, logo, portanto.
DET	Artigo	Um, uma, o, a.
INTJ	Interjeição	Olá, bravo, psiu, ai.
NOUN	Substantivo	Garota, gato, árvore, beleza, coisa, pedra.
NUM	Numeral	Sete, treze, 1, 2, 3, I, II, III, IV.
PART	Partícula	Não.
PRON	Pronome	Eu, ele, ela, você, nós, eles, alguém.
PROPN	Nome próprio	João, Maria, Pedro, Alura, FIAP.
PUNCT	Pontuação.	. (ponto), , (vírgula), ! (exclamação), ? (interrogação).
SCONJ	Conjunção subordinativa	Se, enquanto, que.
SYM	Símbolo	$, %, #, @, +, -, =.
VERB	Verbo	Correr, andar, saltar, comer, pular, estudar.
X	Outros	Qwerty, asddfgghjkuiggv.
SPACE	Espaço entre palavras	

Quadro 2 - Possíveis valores do atributo POS
Fonte: Elaborado pelo autor (2025); adaptado de Raj (2019)

No código-fonte a seguir, é apresentado o uso do POS do spaCy para determinar as classes gramaticais das palavras.

```python
import spacy

nlp = spacy.load("pt_core_news_sm")

texto = (
    "A inteligência artificial, aliada ao Processamento de Linguagem Natural (NLP) transforma dados em compreensão, permitindo que máquinas entendam e se comuniquem com os humanos de forma cada vez mais natural."

doc = nlp(texto)

print(f"{'TOKEN':<20}{'POS':<10}{'TAG'}")
print("=" * 50)

for token in doc:
    if not token.is_punct and not token.is_space:
        print(f"{token.text:<20}{token.pos_:<10}{token.tag_}")
```

Código-fonte 4 - Uso de POS com o spaCy em Python
Fonte: Elaborado pelo autor (2025)

```
TOKEN               POS       TAG
==================================================
A                   DET       DET
inteligência        NOUN      NOUN
artificial          ADJ       ADJ
aliada              VERB      VERB
ao                  ADP       ADP
Processamento       PROPN     PROPN
de                  ADP       ADP
Linguagem           PROPN     PROPN
Natural             PROPN     PROPN
NLP                 PROPN     PROPN
transforma          VERB      VERB
dados               NOUN      NOUN
em                  ADP       ADP
compreensão         NOUN      NOUN
permitindo          VERB      VERB
que                 SCONJ     SCONJ
máquinas            NOUN      NOUN
entendam            VERB      VERB
e                   CCONJ     CCONJ
se                  PRON      PRON
comuniquem          VERB      VERB
com                 ADP       ADP
os                  DET       DET
humanos             NOUN      NOUN
de                  ADP       ADP
forma               NOUN      NOUN
cada                NOUN      NOUN
vez                 ADP       ADP
mais                ADV       ADV
natural             ADJ       ADJ
```

Figura 2 - Saída do código-fonte 4, classes gramaticais das palavras com spaCy
Fonte: Elaborado pelo autor (2025)

3.2.2 Análise sintática com gramática livre de contexto

A diferença entre o **POS tagging** e a **análise sintática com gramáticas livres de contexto (CFG — Context-Free Grammar)** está, principalmente, no nível de profundidade e no tipo de estrutura linguística que cada abordagem busca identificar.

O POS tagging é uma etapa inicial da análise linguística, que consiste em atribuir a cada palavra de uma sentença sua categoria gramatical básica, como substantivo, verbo, adjetivo, advérbio, entre outras. Essa atribuição permite reconhecer o papel morfológico e funcional de cada termo isoladamente, com base em seu contexto imediato. No entanto, essa análise não revela as relações estruturais mais amplas entre as palavras na frase, como sujeito, objeto direto, modificador ou predicado.

Já a análise sintática com gramáticas livres de contexto vai além: ela tenta descrever a **estrutura hierárquica completa da sentença**, organizando os termos em constituintes como sintagmas nominais, verbais e preposicionais. A CFG utiliza regras formais de reescrita — por exemplo, S → NP VP, indicando que uma sentença é composta por um sintagma nominal seguido de um sintagma verbal — para construir uma **árvore sintática,** que representa como as palavras se combinam para formar unidades maiores.

Enquanto o POS tagging apenas rotula as palavras com suas classes gramaticais, a análise sintática com CFG explica como essas palavras se relacionam estruturalmente entre si para formar sentenças válidas na língua.

Em termos práticos, o POS tagging pode ser considerado uma etapa preparatória ou auxiliar da análise sintática, visto que a identificação correta das categorias gramaticais é essencial para aplicar corretamente as regras de uma CFG. No entanto, a análise sintática propriamente dita envolve uma compreensão mais profunda da gramática da língua, permitindo identificar ambiguidades sintáticas, hierarquias de constituintes e até detectar erros gramaticais em uma frase. Portanto, o POS tagging oferece uma visão linear e local da estrutura da frase, enquanto a análise com CFG fornece uma visão global e hierárquica da composição sintática.

Para incorporar as restrições sintáticas — que demonstrem como as palavras se relacionam entre si para construir as frases —, devemos utilizar um algoritmo

específico denominado analisador sintático. Existem diversas maneiras de construir um analisador sintático, como as gramáticas livres de contexto e as gramáticas de rede de transição.

É necessário basearmos o desenvolvimento do analisador sintático em regras gramaticais, como as CFGs, para descrever como frases e sentenças são estruturadas em uma língua. Essas regras são explicitamente definidas por linguistas ou programadores, não sendo aprendidas a partir de dados.

Dessa forma, o analisador (algoritmo) constrói estruturas hierárquicas (como árvores de derivação ou árvores de constituintes), que mostram como palavras em uma sentença se agrupam em frases e como essas frases se conectam. Essa etapa é chamada de construção de árvores sintáticas. Como o analisador sintático segue regras claras e explícitas, é possível entender exatamente por que uma sentença foi analisada de determinada forma — uma característica típica dos sistemas simbólicos que garante determinismo e interpretabilidade.

Para entendermos melhor como funciona o analisador sintático, vamos construir nos mesmo um algoritmo rudimentar utilizando CFG. Primeiramente, precisamos definir a gramática (usaremos a função CFG.fromstring da biblioteca NLTK).

Nessa gramática, é necessário definir:

- S (sentença) é formada por um NP (nome de frase, sintagma nominal) e um VP (verbo de frase, sintagma verbal).
- NP é um determinante (Det) seguido de um substantivo (N).
- VP é um verbo (V) seguido de um NP.

As regras de produção são bem simples:

- Det pode ser "O" ou "A".
- N pode ser "advogado" ou "contrato".
- V é "revisou".

Vamos utilizar o ChartParser do NLTK — que é um tipo de parser que pode lidar com gramáticas como a que definimos — e, em seguida, analisaremos a sentença `['O', 'advogado', 'revisou', 'o', 'contrato']`.

Veja a seguir o programa para implementar o analisador sintático:

```
from nltk import CFG
from nltk import ChartParser

# Definindo uma gramática em formato CFG
grammar = CFG.fromstring("""
  S -> NP VP
  NP -> Det N
  VP -> V NP
  Det -> 'O' | 'A' | 'o' | 'a'
  N -> 'advogado' | 'contrato'
  V -> 'revisou'
""")

# Criando um parser a partir da gramática
parser = ChartParser(grammar)
# Sentença que queremos analisar
sentence = ['O', 'advogado', 'revisou', 'o', 'contrato']

# Gerando a árvore sintática
for tree in parser.parse(sentence):
    tree.pretty_print()
```

Código-fonte 5 - Demonstração de um analisador léxico com NLTK em Python
Fonte: Elaborado pelo autor (2025)

Na figura “Saída do código-fonte 5, árvore sintática com CFG do NTLK”, podemos ver o resultado desse código. Observe como a hierarquia da frase se forma, com o nível mais alto de frase (em S), quebrado imediatamente em dos sintagmas: nominal, mais à esquerda, e verbal, mais à direita. As várias funções gramaticais das palavras são apresentadas na sequência, conforme a hierarquia avança para o nível mais individual (dos tokens), na porção de baixo da figura.

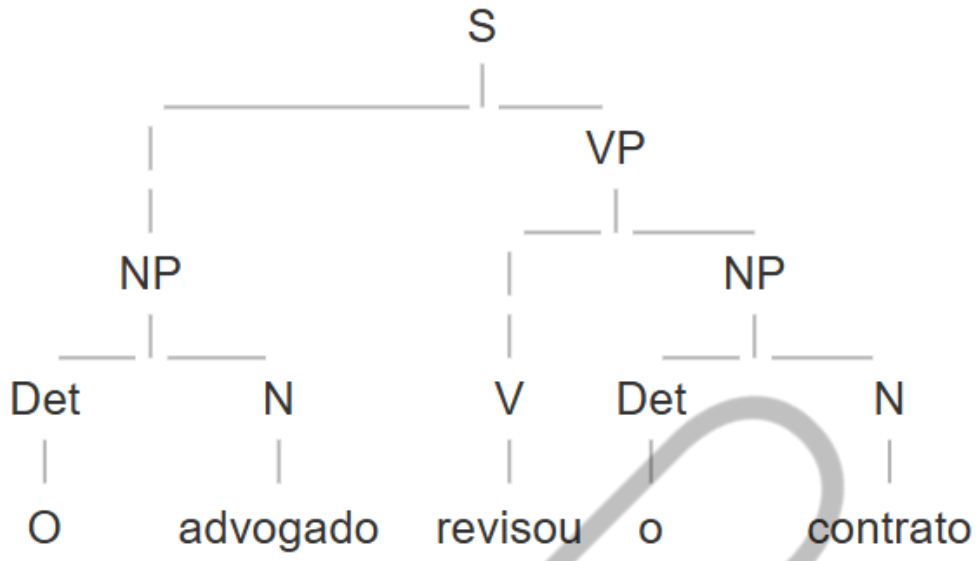


Figura 3 - Saída do código-fonte 5, árvore sintática com CFG do NTLK
Fonte: Elaborado pelo autor (2025)

As gramáticas livres de contexto permitem construir regras gerais, implementado a gramática de uma língua em um código de computador. Entretanto, elas apresentam algumas limitações devido à necessidade de um grande número de regras para capturar toda a complexidade de uma língua.

Importante: parser é um nome muito comum em ciência da computação. No contexto de NLP, um **analisador sintático** (ou *syntactic parser*) interpreta a estrutura gramatical de uma sentença com base em uma gramática formal.

3.2.3 Análise de dependência

A análise de dependência é uma abordagem sintática que representa a estrutura de uma frase com base nas relações diretas entre palavras, priorizando os vínculos gramaticais que conectam termos como sujeito, objeto e modificadores ao verbo principal. Em vez de organizar a sentença em constituintes hierárquicos, como fazem as gramáticas livres de contexto, a análise de dependência foca nas ligações binárias entre um termo regente (geralmente um verbo) e seus dependentes. Por exemplo, na frase “O aluno leu o livro”, a análise de dependência identifica “leu” como o núcleo da oração, ao qual “aluno” está ligado como sujeito e “livro” como objeto direto.

Essa abordagem é utilizada em processamento de linguagem natural por sua capacidade de lidar com línguas de ordem flexível, como o português, além de permitir a construção de representações sintáticas mais diretas e compactas.

O analisador de dependência do spaCy funciona com base em um modelo de Aprendizado de Máquina treinado para prever, a cada etapa, a próxima ação de análise sintática em um sistema de transições. Ele constrói a estrutura de dependência de uma sentença ao aplicar, passo a passo, operações como "shift", "reduce", "left-arc" e "right-arc", até que todas as relações de dependência estejam estabelecidas.

Embora o resultado gerado — uma árvore sintática com relações explícitas entre palavras — seja característico da abordagem simbólica em PLN, o processo pelo qual essa estrutura é produzida não é baseado em regras linguísticas codificadas manualmente. Em vez disso, ele depende de modelos estatísticos treinados em grandes conjuntos de dados anotados.

Por esse motivo, o analisador do spaCy não pode ser considerado puramente simbólico: ele pertence à categoria de métodos híbridos, em que a representação simbólica é gerada por um processo estatístico baseado em aprendizado supervisionado.

Segue o uso do analisador de dependência para plotar a imagem de dependência (arcos de relação entre as palavras):

```python
import spacy
from spacy import displacy

# Carregar modelo de português
nlp = spacy.load("pt_core_news_sm")

# Frase de exemplo
frase = "O aluno leu o livro."

# Processar a frase
doc = nlp(frase)

# Exibir a visualização no navegador
displacy.render(doc, style="dep", jupyter=True)
```

Código-fonte 6 - Análise de dependência com o spaCy em Python
Fonte: Elaborado pelo autor (2025)

Na figura a seguir, observe que “livro” (substantivo, tag NOUN) tem como descendente a palavra “o” (artigo, tag DET) e como ancestral a palavra “leu” (verbo, tag VERB). De maneira geral, as bases das setas indicam **ancestrais** (tokens acima na árvore de dependência), enquanto as pontas das setas indicam **descendentes** (tokens abaixo) de cada palavra em uma frase processada com spaCy.

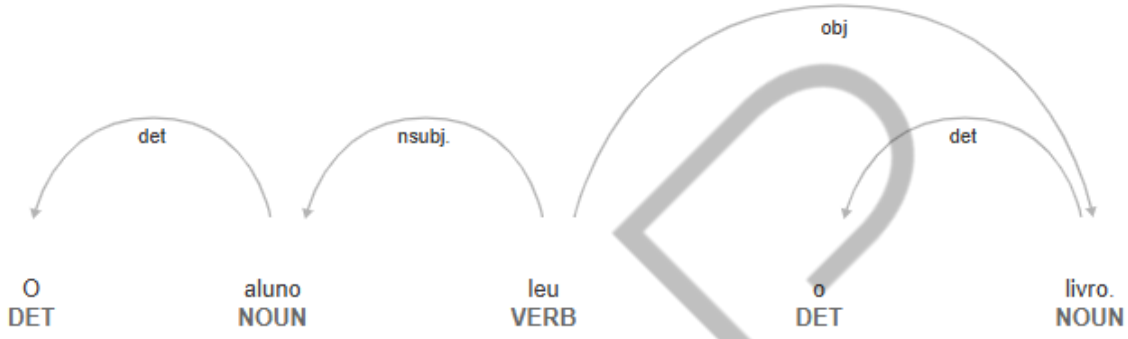


Figura 4 - Saída do código-fonte 6, árvore de dependência com spaCy
Fonte: Elaborado pelo autor (2025)

Podemos alterar o código para imprimir na tela cada um dos ancestrais e descendentes de uma palavra no documento analisado (a frase de exemplo), conforme demonstrado no código-fonte a seguir:

```
# Iterar sobre cada token e exibir ancestrais e descendentes
for token in doc:
    print(f"\nPalavra: {token.text}")

    # Ancestrais
    ancestrais = list(token.ancestors)
    if ancestrais:
        print("   Ancestrais:", ", ".join([a.text for a in
ancestrais]))
    else:
        print("  Ancestrais: Nenhum")

    # Descendentes
    descendentes = list(token.children)
    if descendentes:
        print("   Descendentes:", ", ".join([d.text for d in
descendentes]))
    else:
        print("  Descendentes: Nenhum")
```

Código-fonte 7 - Complemento do código-fonte 6 para imprimir descendentes e ancestrais
Fonte: Elaborado pelo autor (2025)

O resultado está na figura:

```
Palavra: O
  Ancestrais: aluno, leu
  Descendentes: Nenhum

Palavra: aluno
  Ancestrais: leu
  Descendentes: O

Palavra: leu
  Ancestrais: Nenhum
  Descendentes: aluno, livro,

Palavra: o
  Ancestrais: livro, leu
  Descendentes: Nenhum

Palavra: livro
  Ancestrais: leu
  Descendentes: o

Palavra: .
  Ancestrais: leu
  Descendentes: Nenhum
```

Figura 5 - Saída do código-fonte 7, imprimindo os descendentes e ancestrais de cada token
Fonte: Elaborado pelo autor (2025)

As gramáticas livres de contexto permitem construir regras gerais, implementado a gramática de uma língua em um código de computador. Contudo, elas apresentam limitações devido à necessidade de um grande número de regras para capturar toda a complexidade de uma língua.

Importante: no contexto de NLP, um **analisador de dependência** (ou *dependency parser*) interpreta a estrutura de uma sentença com base na relação pai-filho (ancestral-descendente) entre as palavras.

3.3 Ambiguidade sintática

A ambiguidade sintática ocorre quando uma sentença pode ter mais de uma maneira de ser interpretada devido à sua estrutura gramatical. Esse tipo de ambiguidade surge quando diferentes árvores sintáticas podem ser geradas para a mesma sequência de palavras, resultando múltiplos significados possíveis. Por exemplo, na frase "Ela viu o homem com o binóculo", não está claro se "com o binóculo" descreve o homem ou o ato de vê-lo. Resolver a ambiguidade sintática é um dos principais desafios em NLP, exigindo técnicas como análise de dependência,

gramáticas probabilísticas e uso de contexto semântico para identificar a interpretação mais plausível.

Vamos estudar um exemplo. Na frase “Ela viu o homem com binóculo”, a estrutura sintática permite duas interpretações: ela usou um binóculo para ver o homem, ou o homem observado estava com um binóculo. Podemos definir um parser sintático utilizando CFG (conforme o código-fonte “Demonstração de um analisador léxico com NLTK em Python”).

O resultado da árvore sintática é apresentado na figura “Saída do código-fonte 8, demonstração de ambiguidade sintática com CFG usando NLTK”. Nela, observamos que o sintagma verbal (VP) pode ser expandido de duas formas diferentes, resultando em duas análises sintáticas possíveis. Em uma delas, o *chunck* “o homem com binóculo” é tratado no mesmo sintagma nominal (NP) — representada pelo **lado a** da figura. Na outra, “o homem” e “com binóculo” estão separados como artigo (Det) e substantivo (N), e sintagma preposicional (PP), respectivamente (**lado b** da figura).

Dica: no jargão de NLP, um ***chunk*** é um sintagma identificado por uma análise superficial da estrutura gramatical (por exemplo, sintagmas nominais — *Noun Phrases*, NP).

```
from nltk import CFG
from nltk import ChartParser

# Definindo uma gramática em formato CFG
grammar = CFG.fromstring("""
  S -> NP VP
  VP -> V NP | V NP PP
  NP -> Det N | Det N PP | Pron
  PP -> P NP

  Det -> 'o'
  N -> 'homem' | 'binóculo'
  V -> 'viu'
  P -> 'com'
  Pron -> 'ela'
""")
parser = ChartParser(grammar)

# Sentença que queremos analisar
sentence = ['ela', 'viu', 'o', 'homem', 'com', 'o', 'binóculo']
```

```
# Gerando a árvore sintática
for tree in parser.parse(sentence):
    tree.pretty_print()
```

Código-fonte 8 - Demonstração de um analisador léxico com NLTK em Python
Fonte: Elaborado pelo autor (2025)

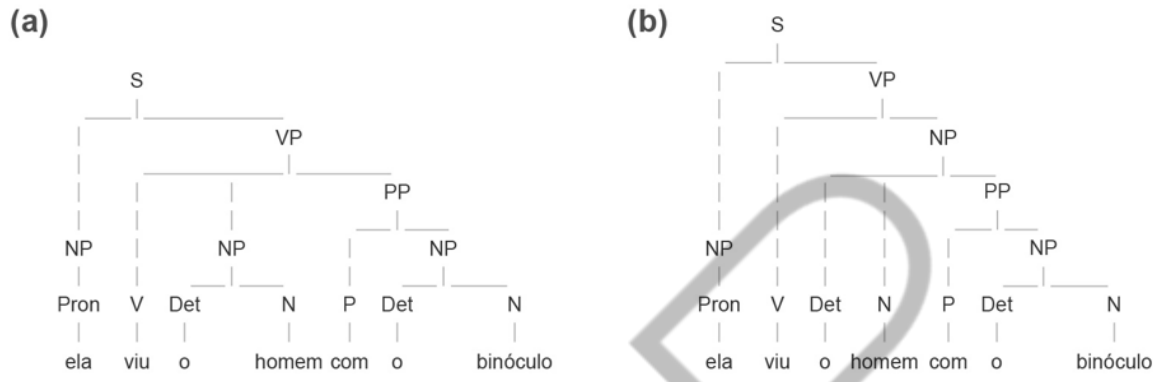


Figura 6 - Saída do código-fonte 8, demonstração de ambiguidade sintática com CFG usando NLTK
Fonte: Elaborado pelo autor (2025)

O spaCy, por padrão, não utiliza gramáticas livres de contexto diretamente como uma ferramenta explícita para análise sintática — ele emprega *parsing* baseado em dependência, com modelos treinados estatisticamente.

É possível observar a estrutura sintática por meio do POS tagging, uma vez que as palavras serão rotuladas em uma determinada classe gramatical:

```
import spacy

nlp = spacy.load("pt_core_news_sm")
sentence = "Ela viu o homem com o binóculo."
doc = nlp(sentence)

for token in doc:
    print(f"{token.text:<12}                    {token.dep_:<10}
{token.head.text:<10} {token.pos_}")
```

Código-fonte 9 - POS tagging e dependências sintáticas com spaCy
Fonte: Elaborado pelo autor (2025)

```
Ela          nsubj      viu        PRON
viu          ROOT       viu        VERB
o            det        homem      DET
homem        obj        viu        NOUN
com          case       binóculo   ADP
o            det        binóculo   DET
binóculo     obl        viu        NOUN
.            punct      viu        PUNCT
```

Figura 7 - Saída do código-fonte 9, demonstração de ambiguidade sintática com spaCy
Fonte: Elaborado pelo autor (2025)

Neste exemplo, o binóculo aparece como um obl (objeto oblíquo) ligado ao verbo viu. Isso sugere que "ela viu com o binóculo" — ou seja, ela usou o binóculo. Se o modelo considerasse "com o binóculo" como uma característica de "homem", o binóculo estaria subordinado ao homem, e não diretamente ao verbo (viu).

FIAP

4 RECONHECIMENTO DE ENTIDADES NOMEADAS

4.1 O que é o reconhecimento de entidades nomeadas?

O reconhecimento de entidades nomeadas (**Named Entity Recognition — NER**, em inglês) é uma tarefa fundamental dentro da área NLP, responsável por identificar e classificar automaticamente informações específicas em um texto. Essas informações são chamadas de “entidades nomeadas” e, geralmente, incluem nomes de pessoas, organizações, locais geográficos, datas, valores monetários, expressões temporais, entre outros.

O principal objetivo do NER é transformar um texto não estruturado em dados estruturados que possam ser analisados de forma mais eficiente por sistemas automatizados. Por exemplo, em uma frase como “Barack Obama nasceu em Honolulu e foi presidente dos Estados Unidos”, um sistema de NER deve ser capaz de identificar "Barack Obama" como uma pessoa, "Honolulu" como uma localidade e "Estados Unidos" como uma organização ou país.

O processo de reconhecimento de entidades pode ser realizado por meio de regras linguísticas (baseadas em gramáticas e dicionários) ou, mais comumente hoje em dia, por meio de técnicas de Aprendizado de Máquina e Redes Neurais Profundas, pois são capazes de capturar padrões complexos na linguagem.

4.2 NER por abordagem simbólica

A abordagem por **regras linguísticas** no reconhecimento de entidades nomeadas baseia-se em identificar padrões de linguagem predefinidos para classificar entidades dentro de um texto. Em vez de usar modelos de Aprendizado de Máquina para aprender esses padrões a partir de grandes volumes de dados, essa abordagem utiliza um conjunto de regras que descrevem explicitamente como identificar diferentes tipos de entidades (como pessoas, lugares, datas etc.). Essas regras podem ser criadas manualmente por linguistas ou especialistas em processamento de linguagem natural, com base nas características da língua e na estrutura sintática do texto.

Primeiramente, para aplicar as regras, é necessário definir quais entidades o sistema deve reconhecer, como nomes de pessoas, locais, datas, organizações e valores. Por exemplo, para reconhecer o nome de uma pessoa, uma regra pode ser criada para identificar palavras com inicial maiúscula, enquanto para reconhecer um local, o sistema pode ser programado para associar palavras como “em” ou “para” seguidas de um nome de cidade ou país. Além disso, listas de palavras-chave, como nomes de países ou empresas, também podem ser utilizadas como referência para identificar essas entidades.

O próximo passo é a criação das regras em si. Essas regras podem se basear em **expressões regulares (regex)**, que são padrões de texto utilizados para buscar correspondências específicas, como datas no formato "dd/mm/aaaa" ou nomes próprios. Outra técnica envolve o uso de padrões **sintáticos** e **morfológicos**, nos quais as entidades são reconhecidas de acordo com a posição na sentença ou a forma gramatical das palavras. Por exemplo, o nome de uma cidade pode ser identificado quando é precedido pela preposição “para” ou “em”. As regras podem ser tanto gerais quanto específicas. Por exemplo, uma regra geral pode identificar qualquer sequência de palavras com maiúsculas como um nome próprio, enquanto uma regra mais específica poderia identificar um nome de pessoa apenas se ele estiver acompanhado de certos títulos ou cargos, como “Dr.” ou “Sr.”. Essas regras podem ser refinadas com o tempo, à medida que novas situações são encontradas.

Uma vez que as regras estão implementadas, o processo de reconhecimento começa com a análise do texto, na qual o sistema percorre cada palavra ou frase, tentando identificar partes que correspondam aos padrões definidos. Quando uma correspondência for encontrada, a entidade é classificada de acordo com o tipo determinado pela regra.

Apesar de suas vantagens, como a **explicabilidade** do processo e o **controle** por parte dos especialistas, a abordagem por regras linguísticas tem algumas limitações. A principal delas é a **escabilidade**: criar regras para cada tipo de entidade ou cada variação linguística pode se tornar um processo demorado e pouco eficiente.

Outro desafio é lidar com as **exceções** e **ambiguidades** que não se encaixam nos padrões predefinidos, o que pode reduzir a robustez do sistema. Por exemplo, a

palavra "Amazon" pode se referir à empresa (Amazon.com) ou à floresta (Amazônia, em inglês). Para lidar com isso, podem ser criadas regras adicionais para o contexto.

4.2.1 NER por regex

Podemos implementar um sistema de NER por meio da criação de regras regex. A seguir, no código-fonte, é demonstrado um exemplo simples disso, com regras para detectar localidade, pessoa e instituição. Execute o código e perceba se ele foi capaz de detectar todas as entidades contidas no texto de exemplo. Como você melhoraria isso?

```
import re

def identificar_localidade(texto):
    # Detecta palavras com inicial maiúscula
    padrao_localidade = r'\b(?:[A-Z][a-z]+(?:\s[A-Z][a-z]+)*)\b'
    localidades = re.findall(padrao_localidade, texto)
    # Lista de palavras-chave
    locais_comuns = ["Brasil", "Estados Unidos", "Paris", "Lisboa", "São Paulo"]
    return [entidade for entidade in localidades if entidade in locais_comuns]


def identificar_pessoa(texto):
    # Regex para encontrar nomes de pessoas com 2 a 4 partes
    padrao_pessoa = r'\b([A-Z][a-z]+(?:\s[A-Z][a-z]+){1,3})\b'
    return re.findall(padrao_pessoa, texto)


def identificar_instituicao(texto):
    # Regex para encontrar instituições
    padrao_instituicao = r'\b(?:Fundação|Universidade|Instituto|Escola|Colégio|Academia|Faculdade)\s+[A-Z][a-z]+(?:\s+[A-Z][a-z]+)*\b'
    return re.findall(padrao_instituicao, texto)


# Função para realizar o NER completo
def NER_por_regex(texto):
    localidades = identificar_localidade(texto)
    instituicoes = identificar_instituicao(texto)
    pessoas = identificar_pessoa(texto)
    return {
```

```
        "Localidades": localidades,
        "Pessoas": pessoas,
        "Instituições": instituicoes
    }

# Exemplo de texto
texto = """
A inteligência artificial começou como um campo experimental na década de 1950 com pioneiros como Allen Newell e Herbert Simon, que fundaram o primeiro laboratório de inteligência artificial na Universidade Carnegie Mellon, e McCarty que juntamente com Marvin Minsky, que fundaram o MIT AI Lab em 1959.
"""

# Executando o NER
resultado = NER_por_regex(texto)

# Exibindo o resultado
print("Entidades Reconhecidas:")
print(f"Localidades: {resultado['Localidades']}")
print(f"Pessoas: {resultado['Pessoas']}")
print(f"Instituições: {resultado['Instituições']}")
```

Código-fonte 10 - Sistema NER com regex em Python
Fonte: Elaborado pelo autor (2025)

4.2.2 NER por análise sintática

No próximo código-fonte, demonstramos o uso de análise sintática para criação de um sistema NER. Aqui, estamos utilizando o POS tagging e a árvore de dependência em spaCy.

```
import spacy

nlp = spacy.load("pt_core_news_md")

def identificar_entidades_por_analise(texto):
    doc = nlp(texto)

    pessoas = []
    localidades = []
    instituicoes = []

    for token in doc:
        # Verifica se o token é um substantivo próprio
        if token.pos_ == "PROPN":

            composto = [token.text]
```

```
            # Análise de entidades baseadas em estrutura
sintática
            for tok in token.rights:
                if tok.pos_ in {"ADP", "PROPN", "DET",
"NOUN"}:
                    composto.append(tok.text)
            entidade = " ".join(composto)

            # Classificar por palavras-chave comuns
            if any(x in entidade for x in ["Universidade",
"Instituto", "Fundação", "Escola", "Faculdade"]):
                instituicoes.append(entidade)
            elif any(x in entidade for x in ["São", "Rio",
"Lisboa", "Paris", "Brasil", "Estados Unidos"]):
                localidades.append(entidade)
            else:
                pessoas.append(entidade)

    return {
        "Pessoas": list(set(pessoas)),
        "Localidades": list(set(localidades)),
        "Instituições": list(set(instituicoes))
    }

# Exemplo de texto
texto = """
A inteligência artificial começou como um campo experimental
na década de 1950 com pioneiros como Allen Newell e Herbert
Simon, que fundaram o primeiro laboratório de inteligência
artificial na Universidade Carnegie Mellon, e McCarty que
juntamente com Marvin Minsky, que fundaram o MIT AI Lab em
1959.
"""

# Executar a análise
resultado = identificar_entidades_por_analise(texto)

# Mostrar o resultado
print("Entidades encontradas:")
for tipo, entidades in resultado.items():
    print(f"{tipo}: {entidades}")
```

Código-fonte 11 - Sistema NER com análise sintática usando spaCy
Fonte: Elaborado pelo autor (2025)

4.2.3 NER por chucking

Chunking, também conhecido como análise sintática rasa (*shallow parsing*), é um processo no qual se identificam e agrupam blocos ou "*chunks*" de palavras em um

texto que formam unidades gramaticais coerentes, como sintagmas nominais, verbais ou preposicionais.

Diferentemente de uma análise sintática completa, o chunking não constrói uma árvore sintática detalhada com todas as relações hierárquicas, mas foca em detectar padrões estruturais básicos com base nas classes gramaticais (como substantivos, verbos e adjetivos) atribuídas aos tokens. Por exemplo, na frase "o cachorro preto", o chunking pode agrupar essas três palavras como um único sintagma nominal.

Essa técnica é amplamente utilizada como etapa intermediária em tarefas de processamento de linguagem natural, como o reconhecimento de entidades nomeadas, visto que auxilia no isolamento de estruturas linguísticas úteis de forma mais simples e eficiente do que uma análise sintática completa.

O NLTK fornece uma função chamada ne_chunk(), que faz o NER com base em chunking gramatical e modelos pré-treinados do tipo árvore de decisão. Esse recurso usa *shallow parsing*, que depende de: tokenização, POS tagging (marcação de classe gramatical) e Chunking (para identificar entidades).

Vejamos um exemplo:

```
import nltk
#nltk.download('averaged_perceptron_tagger_eng')
#nltk.download('maxent_ne_chunker_tab')
#nltk.download('words')
from nltk import word_tokenize, pos_tag, ne_chunk

sentence = "Barack Obama was born in Hawaii."
tokens = word_tokenize(sentence)
tags = pos_tag(tokens)
tree = ne_chunk(tags)
print(tree)
```

Código-fonte 12 - Sistema NER com análise sintática rasa usando NLTK
Fonte: Elaborado pelo autor (2025)

```
(S
  (PERSON Barack/NNP)
  (PERSON Obama/NNP)
  was/VBD
  born/VBN
  in/IN
  (GPE Hawaii/NNP)
  ./.)
```

Figura 8 - Saída do Código-fonte 12, demonstração NER com NLTK
Fonte: Elaborado pelo autor (2025)

Observe que, na saída, existem entidades pré-determinadas marcando certas palavras: PERSON e GPE que representam, respectivamente, as entidades de pessoa e de localidade, marcando as palavras Barack, Obama e Hawaii.

Importante: uma abordagem simbólica robusta deve combinar as técnicas de regex e análise sintática para construir regras de extração poderosas e completas. Para um sistema mais detalhado, o uso de taxonomias e ontologias pode se mostrar um elemento essencial.

4.3 NER por meio da abordagem de Aprendizado de Máquina

A abordagem NER por Aprendizado de Máquina utiliza algoritmos para identificar e classificar automaticamente entidades, como pessoas, lugares e organizações, em textos. Ao contrário das abordagens baseadas em regras, que dependem de padrões linguísticos explícitos, o Aprendizado de Máquina permite que os modelos aprendam as relações entre palavras e seu contexto diretamente a partir de grandes quantidades de dados rotulados.

Modelos como **Redes Neurais Recorrentes (RNNs)** e **Transformers** (como o **BERT**) têm mostrado grande eficácia ao capturar dependências contextuais complexas e nuances linguísticas. No entanto, essa abordagem exige grandes volumes de dados anotados e poder computacional substancial, além de lidar com desafios como ambiguidade linguística e variabilidade nos textos.

O spaCy possui um módulo NER implementado por uma abordagem de Aprendizado de Máquina, baseando-se em modelos estatísticos treinados com dados anotados, em vez de regras fixas ou padrões escritos manualmente. Em outras palavras, o modelo aprende a reconhecer entidades a partir de exemplos fornecidos durante o treinamento. Esses exemplos são trechos de texto nos quais as entidades estão previamente marcadas, como nomes de pessoas, organizações ou localizações. A partir desses dados, o modelo identifica padrões linguísticos e contextuais — como a posição da palavra na frase, os tokens vizinhos, a capitalização, entre outros —, que indicam a presença de uma entidade.

Diferente de um sistema baseado em regras, que depende de instruções explícitas do programador (por exemplo, "se a palavra começar com letra maiúscula e for seguida de 'Inc.', marque como organização"), o NER do spaCy utiliza uma rede

neural que estima probabilidades para identificação possível de entidades, com base nos dados de treinamento. Isso torna o sistema mais flexível, permitindo que ele seja retreinado com dados específicos de determinado domínio, como textos médicos ou jurídicos.

O componente de reconhecimento de entidades nomeadas do spaCy funciona com base em um algoritmo de transição, uma abordagem que processa sequências de tokens tomando decisões passo a passo, de forma incremental. O objetivo do NER é identificar **spans** de tokens contínuos que correspondam a entidades, como pessoas, organizações ou localizações — garantindo que esses spans não se sobreponham.

O algoritmo de transição parte de um estado inicial e, a cada token, decide uma ação que pode indicar o início de uma entidade, sua continuação, seu término ou simplesmente decidir que aquele token não pertence a nenhuma entidade. Essa sequência de decisões é aprendida durante o treinamento e permite ao modelo construir a estrutura final de entidades no texto.

A função de perda utilizada neste componente é otimizada para precisão de entidades inteiras, ou seja, o modelo é penalizado quando erra os limites completos de uma entidade, mesmo que apenas um token do span esteja incorreto. Por isso, esse método pressupõe que as anotações de treinamento tenham uma boa concordância entre os anotadores nos limites das entidades. Caso contrário, o desempenho pode ser prejudicado.

Além disso, o algoritmo assume que as informações mais relevantes para identificar uma entidade estão próximas do seu início. Isso significa que ele tende a funcionar melhor com entidades curtas e com pistas contextuais logo nos primeiros tokens.

Em tarefas nas quais as entidades são longas e os elementos mais importantes estão no meio ou no final do span, esse tipo de abordagem pode não ser a mais adequada.

Dica: em processamento de linguagem natural, **span** é um termo utilizado para se referir a um **trecho contínuo de texto**, composto por um ou mais tokens sem, necessariamente, conter inter-relação gramatical, como ocorre em um *chunck*.

A seguir, demonstramos o uso do módulo NER do spaCy para processar a mesma frase que utilizamos nos códigos-fonte “Sistema NER com regex em Python” e “Sistema NER com análise sintática usando spaCy”.

```
import spacy

nlp = spacy.load("pt_core_news_sm")

# Exemplo de texto
texto = """
A inteligência artificial começou como um campo experimental na década de 1950 com pioneiros como Allen Newell e Herbert Simon, que fundaram o primeiro laboratório de inteligência artificial na Universidade Carnegie Mellon, e McCarty que juntamente com Marvin Minsky, que fundaram o MIT AI Lab em 1959.
"""

# Processa o texto com o modelo
doc = nlp(texto)

# Exibe as entidades reconhecidas
print("Entidades reconhecidas:")
for ent in doc.ents:
    print(f"{ent.text} ({ent.label_})")
```

Código-fonte 13 - Sistema NER com Aprendizado de Máquina usando o spaCy
Fonte: Elaborado pelo autor (2025)

```
Entidades reconhecidas:
Allen Newell (PER)
Herbert Simon (PER)
Universidade Carnegie Mellon (LOC)
McCarty (PER)
Marvin Minsky (PER)
MIT AI (MISC)
```

Figura 9 - Saída do Código-fonte 13, demonstração NER por Aprendizado de Máquina com spaCy
Fonte: Elaborado pelo autor (2025)

A figura anterior ilustra o resultado do código-fonte. Observe que as entidades encontradas estão marcadas com PER, LOC e MISC.

O significado dessas tags é resumido no quadro:

Entidade	Descrição
PERSON	Pessoas, incluindo fictícias.
NORP	Nacionalidades ou grupos religiosos ou políticos.
FAC	Edifícios, aeroportos, rodovias, pontes etc.
ORG	Empresas, agências, instituições etc.
GPE	Países, cidades, estados.
LOC	Locais que não são GPE, cadeias de montanhas, rios etc.
PRODUCT	Objetos, veículos, comidas etc. (não serviços).
EVENT	Furacões nomeados, batalhas, eventos esportivos etc.
WORK_OF_ART	Títulos de livros, músicas etc.
LAW	Documentos nomeados que se tornaram leis.
LANGUAGE	Qualquer idioma nomeado.
DATE	Datas ou períodos absolutos ou relativos.
TIME	Horários menores que um dia.
PERCENT	Porcentagem, incluindo “%”.
MONEY	Valores monetários, incluindo unidade.
QUANTITY	Medidas, como peso ou distância.
ORDINAL	“primeiro”, “segundo” etc.
CARDINAL	Números que não se enquadram em outro tipo.

Quadro 3 - Entidades treinadas no módulo de NER do spaCy
Fonte: Elaborado pelo autor (2025); adaptado da documentação do spaCy (2025)

5 ONTOLOGIAS

5.1 O que são ontologias?

Uma ontologia, em termos filosóficos, é o estudo do ser, ou seja, a análise dos tipos e das categorias de entidades que existem no mundo. Em Ciência da Computação, uma ontologia pode ser definida como uma representação explícita de um conjunto de conceitos dentro de um domínio e as relações entre esses conceitos. Ela busca responder perguntas como “o que é um objeto?”, “como conceitos se relacionam?” e “quais são as propriedades de um conceito?”. Uma ontologia é frequentemente composta de classes (ou conceitos), instâncias (ou indivíduos), relações (ou propriedades) e restrições.

Na prática, ontologia refere-se a um modelo formal que define um conjunto de conceitos e as relações entre esses conceitos em um domínio específico. Em termos mais simples, é uma maneira estruturada de representar o conhecimento sobre um determinado tema de forma que os sistemas computacionais possam processá-lo e usá-lo para várias finalidades, como análise, raciocínio ou integração de dados.

As ontologias são fundamentais na **representação do conhecimento**, sendo utilizadas para criar modelos estruturados e explícitos de um determinado domínio do conhecimento, como a medicina, a biologia ou a engenharia. Por exemplo, no domínio da biologia, uma ontologia pode definir claramente conceitos como "espécie", "gênero", "habitat" e suas inter-relações. Dessa forma, ela não apenas organiza as informações, mas também permite uma análise mais profunda e eficiente do conhecimento neste domínio. Além disso, ela facilita a comunicação entre especialistas, pois oferece uma linguagem comum para descrever os mesmos conceitos e suas inter-relações.

As ontologias também desempenham um papel crucial na combinação de informações provenientes de diferentes fontes, auxiliando na **integração de dados**. Por exemplo, dados de diferentes sistemas ou bases de dados que utilizam terminologias ou formatos distintos podem ser unificados por meio de uma ontologia, possibilitando que esses dados sejam interpretados de forma coerente. Isso é particularmente útil em cenários complexos — como os sistemas empresariais, que

possuem informações dispersas em vários departamentos ou até entre diferentes organizações.

Outro campo importante de aplicação é a **Inteligência Artificial**. As ontologias são utilizadas em sistemas de raciocínio automático, nos quais a IA pode fazer inferências baseadas em um conjunto de conhecimentos estruturados. Por exemplo, um sistema de IA que usa uma ontologia de saúde pode, ao aprender novos dados sobre sintomas e diagnósticos, inferir possíveis novas doenças ou tratamentos, baseando-se nas relações e regras definidas na ontologia. Assim, ela não apenas organiza a informação, como também facilita a capacidade do sistema de gerar novos insights e tomar decisões de forma autônoma.

Na **web semântica**, as ontologias desempenham um papel fundamental ao melhorar a busca e análise de informações na internet. A web semântica busca oferecer um entendimento mais profundo dos conteúdos disponíveis online, conectando dados de maneira mais inteligente. Por exemplo, quando você faz uma busca no Google, o mecanismo utiliza as ontologias para entender melhor o contexto da sua consulta e fornecer os resultados mais relevantes. Se você procurar por “restaurantes italianos”, uma ontologia pode ajudar a classificar e interpretar as informações de maneira a mostrar apenas os resultados que são realmente relevantes para sua pesquisa, como opções perto de você ou aquelas com base em avaliações e categorias.

No campo do NLP, ontologias não são apenas usadas para **desambiguar** palavras, mas também para facilitar tarefas complexas como extração de informações, reconhecimento de entidades nomeadas, análise de sentimentos, geração de texto e tradução automática.

As ontologias possuem um tipo de **ontologia de domínio**, onde palavras ou conceitos relevantes para aquele domínio específico — como medicina, direito ou esportes — são bem definidos, com suas relações e propriedades detalhadas.

Observação: apesar de parecidas, **ontologia** e **antologia** significam coisas diferentes; ontologia é um modelo formal de representação do conhecimento, enquanto antologia é uma coleção organizada de obras literárias.

5.2 Construindo ontologias

Construir uma ontologia envolve uma série de etapas estruturadas para garantir que o conhecimento de um domínio específico seja representado de forma clara e coerente. O primeiro passo é identificar o domínio de interesse e os objetivos da ontologia, ou seja, o que se deseja alcançar com ela, como integrar dados ou melhorar a busca por informações. Em seguida, é importante identificar os principais conceitos (ou classes) do domínio e suas relações, como hierarquias ou dependências entre eles. A partir disso, torna-se necessário definir as propriedades ou atributos desses conceitos, bem como as regras que determinam como eles interagem. Após o mapeamento das entidades e suas relações, essas informações precisam ser representadas de maneira formal, utilizando linguagens como OWL ou RDF.

Uma ontologia deve ser composta por:

- **Definição de conceitos (ou classes):** identificação e definição das entidades do domínio.
  - Exemplo: "Pessoa", "Veículo", "Produto".
- **Relações entre conceitos:** descrição das conexões ou associações entre os conceitos.
  - Exemplo: "Uma Pessoa dirige um Veículo".
- **Propriedades (ou atributos):** características ou atributos dos conceitos.
  - Exemplo, "Idade" de uma "Pessoa", "Cor" de um "Veículo".
- **Instâncias:** exemplos ou dados específicos que instanciam os conceitos definidos na ontologia.
  - Exemplo: "João" é uma instância da classe "Pessoa".

Uma vez que as bases conceituais da ontologia estão definidas, o próximo desafio é garantir a consistência e a usabilidade do modelo criado. A ontologia deve ser capaz de evoluir e se adaptar ao longo do tempo, já que os domínios de conhecimento estão sempre sujeitos a mudanças e novas descobertas.

Para isso, a manutenção contínua da ontologia é essencial. Isso pode incluir a adição de novos conceitos, o ajuste de relações ou a refinação das definições

existentes. Além disso, é crucial garantir que a ontologia seja interoperável com outras ontologias ou sistemas de informações. Isso requer o uso de mecanismos de alinhamento ou mapeamento, que permitam integrar diferentes fontes de conhecimento sem perda de significado. Para garantir essa interoperabilidade, a utilização de padrões e vocabulários amplamente aceitos no campo, como SKOS ou Dublin Core, pode ser uma estratégia eficiente.

Por fim, a validação da ontologia é um passo fundamental para confirmar que ela reflete de maneira precisa e eficiente o domínio de conhecimento. Essa validação pode ser realizada por meio de revisões por especialistas no assunto, testes de aplicação prática ou análise de sua capacidade de responder a perguntas e fornecer resultados úteis. Assim, garante-se que a antologia não seja apenas teoricamente sólida, mas também funcional no uso cotidiano.

É importante ressaltar que OWL, RDF, SKOS e Dublin Core são diferentes linguagens. Embora sigam padrões utilizados na representação de conhecimento, cada uma possui finalidades e características distintas.

O **RDF** (Resource Description Framework) é uma linguagem padrão para descrever recursos na web de maneira simples e estruturada — baseada em triplas ordenadas (sujeito-predicado-objeto) —, permitindo a modelagem de dados de forma que possam ser facilmente compartilhados e processados por máquinas. Trata-se da fundação sobre a qual muitas outras tecnologias semânticas, como OWL e SKOS, são construídas.

Já o **OWL** (Web Ontology Language) é uma linguagem mais avançada e expressiva, padronizada e definida pelo W3C, para criar e compartilhar ontologias na web. Enquanto o RDF permite a descrição de dados de forma simples, o OWL vai além, permitindo a definição de classes complexas, propriedades e restrições que facilitam a representação de um domínio de conhecimento de maneira detalhada e formalizada, permitindo raciocínios automáticos sobre os dados.

O **SKOS** (Simple Knowledge Organization System), por sua vez, é um vocabulário utilizado para representar esquemas de classificação e taxonomias, como ontologias simples e outros sistemas de organização de conhecimento. Ao contrário do OWL, que pode ser utilizado para modelar ontologias complexas, o SKOS é mais

simples e focado em relações hierárquicas e de equivalência entre termos, o que o torna adequado para representar vocabulários controlados.

Por último, o **Dublin Core** é um vocabulário de metadados amplamente utilizado para descrever recursos digitais, como documentos, imagens ou vídeos, de maneira padronizada, com foco em informações como título, autor, data de criação, entre outros. Ao contrário do OWL, o Dublin Core é mais orientado a descrever os recursos de maneira geral e não se destina a modelar relações complexas ou lógicas entre conceitos.

Para entender um pouco melhor como essas ideias funcionam, vamos trabalhar em um exemplo prático. No código-fonte a seguir, é apresentada uma ontologia construída com RDF/OWL. Ela é definida como um XML (Extensible Markup Language), uma linguagem de marcação que fornece regras para definir quaisquer dados, semelhante ao HTML, que é usado para construir páginas web.

```
<?xml version="1.0" encoding="UTF-8"?>
<rdf:RDF
    xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
    xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#"
    xmlns:owl="http://www.w3.org/2002/07/owl#"
    xmlns:xsd="http://www.w3.org/2001/XMLSchema#"
    xmlns:ex="http://www.example.org/ontologia_futebol#">

  <!-- Classes -->
  <owl:Class
rdf:about="http://www.example.org/ontologia_futebol#Jogador">
    <rdfs:label>Jogador</rdfs:label>
  </owl:Class>

  <owl:Class
rdf:about="http://www.example.org/ontologia_futebol#Time">
    <rdfs:label>Time</rdfs:label>
  </owl:Class>

  <owl:Class
rdf:about="http://www.example.org/ontologia_futebol#Estádio">
    <rdfs:label>Estádio</rdfs:label>
  </owl:Class>

  <!-- Propriedades de Objeto -->
  <owl:ObjectProperty
rdf:about="http://www.example.org/ontologia_futebol#jogaEm">
```

```
    <rdfs:domain
rdf:resource="http://www.example.org/ontologia_futebol#Jogado
r"/>
    <rdfs:range
rdf:resource="http://www.example.org/ontologia_futebol#Time"/
>
  </owl:ObjectProperty>

  <owl:ObjectProperty
rdf:about="http://www.example.org/ontologia_futebol#temJogado
r">
    <rdfs:domain
rdf:resource="http://www.example.org/ontologia_futebol#Time"/
>
    <rdfs:range
rdf:resource="http://www.example.org/ontologia_futebol#Jogado
r"/>
  </owl:ObjectProperty>

  <owl:ObjectProperty
rdf:about="http://www.example.org/ontologia_futebol#jogaNo">
    <rdfs:domain
rdf:resource="http://www.example.org/ontologia_futebol#Time"/
>
    <rdfs:range
rdf:resource="http://www.example.org/ontologia_futebol#Estádi
o"/>
  </owl:ObjectProperty>

  <owl:DatatypeProperty
rdf:about="http://www.example.org/ontologia_futebol#localizad
oEm">
    <rdfs:domain
rdf:resource="http://www.example.org/ontologia_futebol#Estádi
o"/>
    <rdfs:range
rdf:resource="http://www.w3.org/2001/XMLSchema#string"/>
  </owl:DatatypeProperty>

  <!-- Propriedades de Dados -->
  <owl:DatatypeProperty
rdf:about="http://www.example.org/ontologia_futebol#nome">
    <rdfs:domain
rdf:resource="http://www.example.org/ontologia_futebol#Jogado
r"/>
    <rdfs:range
rdf:resource="http://www.w3.org/2001/XMLSchema#string"/>
  </owl:DatatypeProperty>

  <owl:DatatypeProperty
rdf:about="http://www.example.org/ontologia_futebol#idade">
```

```
    <rdfs:domain
rdf:resource="http://www.example.org/ontologia_futebol#Jogado
r"/>
    <rdfs:range
rdf:resource="http://www.w3.org/2001/XMLSchema#integer"/>
  </owl:DatatypeProperty>

  <owl:DatatypeProperty
rdf:about="http://www.example.org/ontologia_futebol#fundacao"
>
    <rdfs:domain
rdf:resource="http://www.example.org/ontologia_futebol#Time"/
>
    <rdfs:range
rdf:resource="http://www.w3.org/2001/XMLSchema#integer"/>
  </owl:DatatypeProperty>

  <owl:DatatypeProperty
rdf:about="http://www.example.org/ontologia_futebol#capacidad
e">
    <rdfs:domain
rdf:resource="http://www.example.org/ontologia_futebol#Estádi
o"/>
    <rdfs:range
rdf:resource="http://www.w3.org/2001/XMLSchema#integer"/>
  </owl:DatatypeProperty>

  <!-- Instâncias de Jogadores -->
  <rdf:Description
rdf:about="http://www.example.org/ontologia_futebol#Cristiano
_Ronaldo">
    <rdf:type
rdf:resource="http://www.example.org/ontologia_futebol#Jogado
r"/>
    <ex:nome
rdf:datatype="http://www.w3.org/2001/XMLSchema#string">Cristi
ano Ronaldo</ex:nome>
    <ex:idade
rdf:datatype="http://www.w3.org/2001/XMLSchema#integer">38</e
x:idade>
    <ex:jogaEm
rdf:resource="http://www.example.org/ontologia_futebol#Manche
ster_United"/>
  </rdf:Description>

  <rdf:Description
rdf:about="http://www.example.org/ontologia_futebol#Lionel_Me
ssi">
    <rdf:type
rdf:resource="http://www.example.org/ontologia_futebol#Jogado
r"/>
```

```
    <ex:nome
rdf:datatype="http://www.w3.org/2001/XMLSchema#string">Lionel
Messi</ex:nome>
    <ex:idade
rdf:datatype="http://www.w3.org/2001/XMLSchema#integer">36</e
x:idade>
    <ex:jogaEm
rdf:resource="http://www.example.org/ontologia_futebol#Paris_
Saint_Germain"/>
  </rdf:Description>

  <!-- Instâncias de Times -->
  <rdf:Description
rdf:about="http://www.example.org/ontologia_futebol#Mancheste
r_United">
    <rdf:type
rdf:resource="http://www.example.org/ontologia_futebol#Time"/
>
    <ex:nome
rdf:datatype="http://www.w3.org/2001/XMLSchema#string">Manche
ster United</ex:nome>
    <ex:fundacao
rdf:datatype="http://www.w3.org/2001/XMLSchema#integer">1878<
/ex:fundacao>
  </rdf:Description>

  <rdf:Description
rdf:about="http://www.example.org/ontologia_futebol#Paris_Sai
nt_Germain">
    <rdf:type
rdf:resource="http://www.example.org/ontologia_futebol#Time"/
>
    <ex:nome
rdf:datatype="http://www.w3.org/2001/XMLSchema#string">Paris
Saint-Germain</ex:nome>
    <ex:fundacao
rdf:datatype="http://www.w3.org/2001/XMLSchema#integer">1970<
/ex:fundacao>
  </rdf:Description>

</rdf:RDF>
```

Código-fonte 14 - Ontologia criada em RDF/OWL para o domínio esportivo do tipo futebol
Fonte: Elaborado pelo autor (2025)

Copie o código e salve em um arquivo no formato `.owl`. Agora, vamos utilizar Python para ler esse arquivo. Primeiro, precisamos instalar a biblioteca Python utilizada na leitura desse formato:

```
pip install rdflib
```

Comando de prompt 1 - Instalação do rdflib
Fonte: Elaborado pelo autor (2025)

No código-fonte a seguir, realizaremos a leitura e, em seguida, iremos imprimir na tela as classes, instâncias e propriedades definidas na ontologia.

```
from rdflib import Graph, RDF, OWL

g = Graph()
g.parse("ontologia_futebol.owl", format="xml")

# Pega todas as classes definidas na ontologia
classes = set(g.subjects(RDF.type, OWL.Class))

# Para cada classe, pega os indivíduos (sujeitos que são do tipo da classe)
individuos = set()
for cls in classes:
    for inst in g.subjects(RDF.type, cls):
        individuos.add(inst)

print("Classes encontradas:")
for c in classes:
    print(c)

print("\nIndivíduos encontrados (somente instâncias das classes):")
for i in individuos:
    print(i)

print("\nPropriedades encontradas na ontologia:")
for prop in g.subjects(RDF.type, OWL.ObjectProperty):
    print(f"ObjectProperty: {prop}")

for prop in g.subjects(RDF.type, OWL.DatatypeProperty):
    print(f"DatatypeProperty: {prop}")
```

Código-fonte 15 - Abrindo uma ontologia com o rdflib em Python
Fonte: Elaborado pelo autor (2025)

O resultado é apresentado na figura:

```
Classes encontradas:
http://www.example.org/ontologia_futebol#Estádio
http://www.example.org/ontologia_futebol#Time
http://www.example.org/ontologia_futebol#Jogador

Indivíduos encontrados (somente instâncias das classes):
http://www.example.org/ontologia_futebol#Lionel_Messi
http://www.example.org/ontologia_futebol#Manchester_United
http://www.example.org/ontologia_futebol#Cristiano_Ronaldo
http://www.example.org/ontologia_futebol#Paris_Saint_Germain

Propriedades encontradas na ontologia:
ObjectProperty: http://www.example.org/ontologia_futebol#jogaEm
ObjectProperty: http://www.example.org/ontologia_futebol#temJogador
ObjectProperty: http://www.example.org/ontologia_futebol#jogaNo
DatatypeProperty: http://www.example.org/ontologia_futebol#localizadoEm
DatatypeProperty: http://www.example.org/ontologia_futebol#nome
DatatypeProperty: http://www.example.org/ontologia_futebol#idade
DatatypeProperty: http://www.example.org/ontologia_futebol#fundacao
DatatypeProperty: http://www.example.org/ontologia_futebol#capacidade
```

Figura 10 - Saída do código-fonte 15, ontologia para domínio esportivo
Fonte: Elaborado pelo autor (2025)

Note que os resultados são marcados como se fossem links da internet. O termo técnico para isso é URI (Uniform Resource Identifier). O URI é um identificador padrão utilizado para nomear recursos na web ou em sistemas de informação de forma única e global. Ele serve para identificar qualquer coisa — como uma página, um documento, uma ontologia, uma classe ou uma instância — por meio de uma sequência de caracteres que segue uma sintaxe específica.

No contexto da web semântica e ontologias, URIs garantem que cada conceito, propriedade ou indivíduo tenha um endereço único que possa ser referenciado e compartilhado entre diferentes sistemas, facilitando a interoperabilidade e a integração de dados.

5.3 Integrando ontologias e ferramentas de NLP

Nesta seção, vamos reunir os conceitos aprendidos e verificar como um sistema prático pode utilizar uma ontologia para validar entidades extraídas com NER. Ainda com base na ontologia do domínio esportivo do exemplo anterior, iremos implementar o seguinte código-fonte:

```
import spacy
from rdflib import Graph, Namespace, Literal
from rdflib.namespace import XSD

# Carrega o modelo de linguagem
nlp = spacy.load("pt_core_news_sm")

texto = """
Cristiano Ronaldo marcou um gol para o Manchester United no
estádio Old Trafford. Lionel Messi joga no Paris Saint-Germain.
"""

doc = nlp(texto)
entidades = [ent.text for ent in doc.ents]
print("Entidades extraídas pelo spaCy:", entidades)

# Carrega a ontologia
g = Graph()
g.parse("ontologia_futebol.owl", format="xml")

# Define namespace usado na ontologia
ex = Namespace("http://www.example.org/ontologia_futebol#")

def entidade_esta_na_ontologia(entidade):
    for s, p, o in g.triples((None, ex.nome, None)):
        if str(o).lower() == entidade.lower():
            return True, s
    return False, None

for ent in entidades:
    achou, uri = entidade_esta_na_ontologia(ent)
    if achou:
        print(f"A entidade '{ent}' está na ontologia com URI:
{uri}")
    else:
        print(f"A entidade '{ent}' NÃO está na ontologia.")
```

Código-fonte 16 - Validando entidades extraídas com NER do spaCy através de uma ontologia
Fonte: Elaborado pelo autor (2025)

Resultado do código-fonte:

```
Entidades extraídas pelo spaCy: ['Cristiano Ronaldo', 'Manchester United', 'Old Trafford', 'Lionel Messi', 'Paris Saint-Germain']
A entidade 'Cristiano Ronaldo' está na ontologia com URI: http://www.example.org/ontologia_futebol#Cristiano_Ronaldo
A entidade 'Manchester United' está na ontologia com URI: http://www.example.org/ontologia_futebol#Manchester_United
A entidade 'Old Trafford' NÃO está na ontologia.
A entidade 'Lionel Messi' está na ontologia com URI: http://www.example.org/ontologia_futebol#Lionel_Messi
A entidade 'Paris Saint-Germain' está na ontologia com URI: http://www.example.org/ontologia_futebol#Paris_Saint_Germain
```

Figura 11 - Saída do código-fonte 16, integração entre NER e ontologia
Fonte: Elaborado pelo autor (2025)

Observe que há uma entidade extraída que não foi definida dentro da ontologia. Você consegue pensar em uma forma de aumentar a ontologia automaticamente, à

medida que novas entidades relacionadas ao domínio de trabalho forem descobertas? Tente utilizar todos os conceitos aprendidos neste capítulo para fazer isso!

5.4 Web semântica

A web semântica é uma extensão da web tradicional, que visa tornar a informação na internet compreensível não apenas para humanos, mas também para máquinas. Por meio do uso de padrões, linguagens e ontologias — como RDF, OWL e SPARQL —, a web semântica organiza dados com significado explícito, permitindo que computadores interpretem, integrem e relacionem informações de forma inteligente. Isso possibilita buscas mais precisas, automação de tarefas complexas e a criação de aplicações capazes de raciocinar sobre os dados, promovendo uma internet mais conectada, inteligente e útil.

Para exemplificar, na figura “Exemplo de aplicação de web semântica na construção de páginas HTML”, vemos um arquivo HTML que segue as diretrizes da web semântica e um que não segue:

FIAP

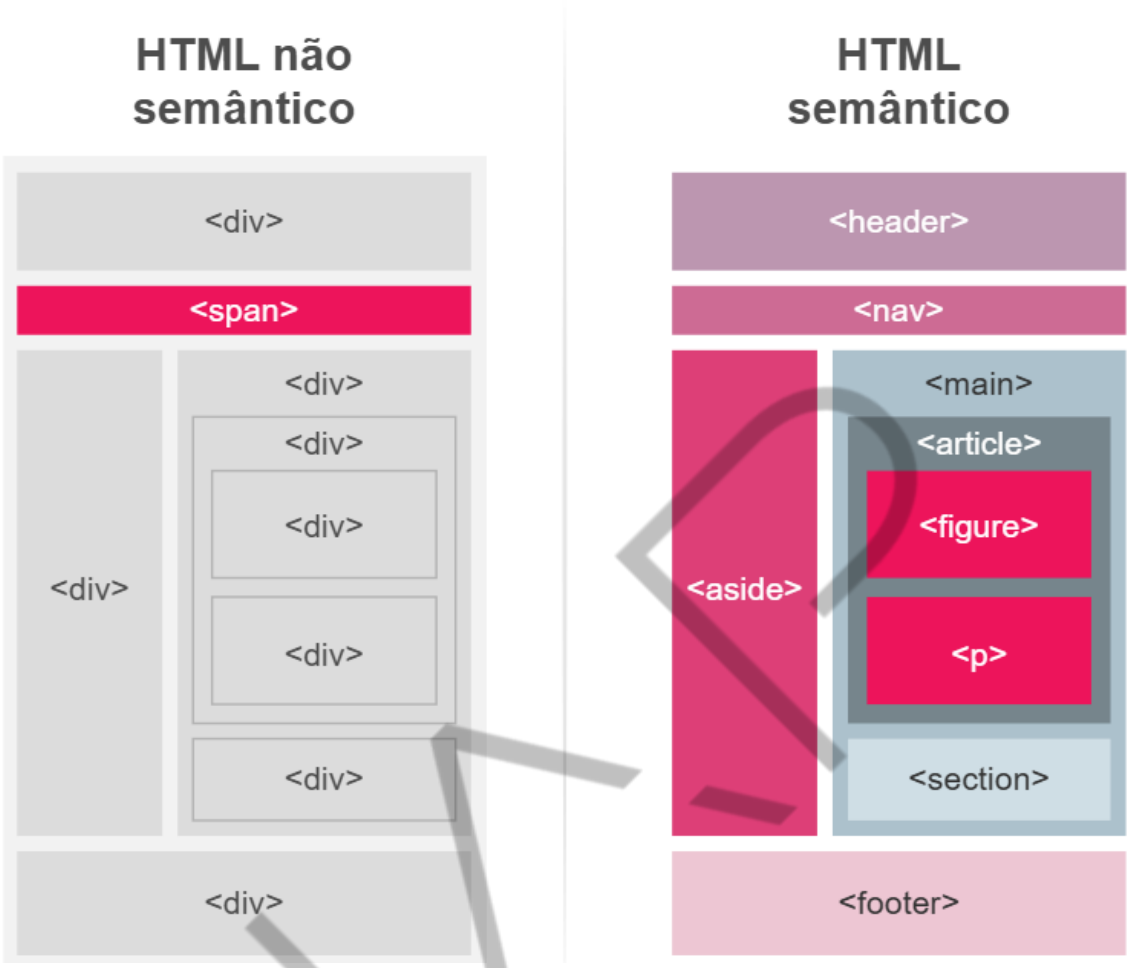


Figura 12 - Exemplo de aplicação de web semântica na construção de páginas HTML
Fonte: Google imagens (2025), adaptado por FIAP (2025)

A web semântica complementa técnicas de NLP e **web scraping** (raspagem de dados) ao estruturar e organizar dados extraídos da web de forma que sejam facilmente interpretáveis por máquinas.

Enquanto o web scraping coleta informações brutas de páginas web — muitas vezes em texto não estruturado, ou seja, texto sem marcações com tags semânticas —, o NLP é utilizado para entender, extrair e classificar entidades e conceitos desse texto.

A web semântica entra em cena para garantir padrões de marcações auxiliares nos documentos que facilitem a recuperação da informação. Ela também ajuda a transformar e validar esses dados extraídos e interpretados em formatos padronizados, usando ontologias e taxonomias, o que possibilita a integração, o compartilhamento e a reutilização inteligente da informação.

Assim, combinar NLP, web scraping e web semântica potencializa a construção de sistemas capazes de capturar, compreender e usar o conhecimento disponível na internet de maneira mais eficiente e automatizada.

FIAP

REFERÊNCIAS

COPPIN, Ben. **Inteligência Artificial**. 1. Ed. LTC, 2010.

DOCUMENTAÇÃO da biblioteca de processamento de linguagem natural em Python. **Spacy**, 2025. Disponível em: <https://spacy.io/>. Acesso em: 27 jun. 2025.

DOCUMENTAÇÃO da biblioteca processamento de linguagem natural em Python. **NLKT**, 2025. Disponível em: <https://www.nltk.org/>. Acesso em: 27 jun. 2025.

FEIGENBAUM, Edward. Some challenges and grand challenges for computational intelligence. **Journal of the ACM (JACM)**, Volume 50, Issue 1, 2003.

JURAFSKY, Dan; MARTIN, James. Speech and Language Processing. 3. Ed. Stanford, 12 de janeiro de 2025. Disponível em: <https://web.stanford.edu/~jurafsky/slp3/>. Acesso em: 27 jun. 2025.

POLLOCK, Jeffrey. **Web Semântica para Leigos**. 1. Ed. Alta Books, 2010.

RAJ, Sumit. **Construindo Chatbots com Python:** Usando Natural Language Processing e Machine Learning. 1. Ed. Novatec, 2019.

RICH, Elaine. **Inteligência Artificial**. 1. Ed. McGraw-Hill, 1988.

WINSTON, Patrick. **Inteligência Artificial**. 1. Ed. LTC, 1988.